```
#-------------------------------------------------------------------------
# Makefile for antBASIC on the Raspberry Pi
#
# NOTICE  You need to pre-install "libreadline-dev", "enscript" and
#         "ghostscript (includes ps2pdf)" packages before "make std|max".
#-------------------------------------------------------------------------
# PREP    $ sudo apt install libreadline-dev atp ghostscript
#-------------------------------------------------------------------------
# BUILD   $ make                --> main applications only
#         $ make std            --> main applications plus PDFs
#         $ make max            --> main and test applications plus PDFS
# INSTALL $ sudo make install   --> install main applications and MAN file
#-------------------------------------------------------------------------
# BAREMETALHACK.COM --> public domain
#-------------------------------------------------------------------------

CFLAGS := -Wall -Wextra
CFLAGS += -DREADLINE ### Build antBASIC with readline extension
CFLAGS += -DPI ### Build antBASIC targeted for Raspberry Pi
# CFLAGS += -DDEBUG ### Turn on in the case of debugging
LDLIBS := -lreadline ### Link with GNU Readline library
ESFLAGS := -M A4 # Page size = A4
ESFLAGS += -2r -C -j --header='$$n|$$D{%F}|Page $$% of $$=' -p -
PSFLAGS := -sPAPERSIZE=a4
GROFFFLAGS := -mandoc -Tps
GROFFFLAGS += -dpaper=a4 ### Paper size
TARGET_BIN := /usr/local/bin
TARGET_MAN := /usr/local/share/man/man1
TARGET_DOC := ./DOCUMENTS

HEADERS = byteword.h token.h bcode.h program.h container.h function.h eval.h basic.h debug.h escape.h
APPS = antbasic antcalib
TESTAPPS = test_token test_bcode test_prog test_eval test_assign
PDFS = $(TARGET_DOC)/antbasic_src.pdf $(TARGET_DOC)/antbasic_man.pdf

.SECONDEXPANSION:

min: antbasic antcalib
std: antbasic antcalib pdfs
max: test_token test_bcode test_prog test_eval test_assign antbasic antcalib pdfs

test_token : $$@.c $(HEADERS) token.c
        gcc $(CFLAGS) -o $@ $(filter %.c, $^)

test_bcode: $$@.c $(HEADERS) token.c bcode.c
        gcc $(CFLAGS) -o $@ $(filter %.c, $^)

test_prog: $$@.c $(HEADERS) token.c bcode.c program.c
        gcc $(CFLAGS) -o $@ $(filter %.c, $^)

test_eval: $$@.c $(HEADERS) token.c bcode.c container.c function.c eval.c pi_gpio.c
        gcc $(CFLAGS) -o $@ $(filter %.c, $^)

test_assign: $$@.c $(HEADERS) token.c bcode.c container.c function.c eval.c pi_gpio.c
        gcc $(CFLAGS) -o $@ $(filter %.c, $^)

antbasic: $(HEADERS) main.c token.c bcode.c program.c container.c function.c eval.c basic.h basic.c pi_gpio.c main.c
        gcc $(CFLAGS) -o $@ $(filter %.c, $^) -lreadline

antcalib: antcalib.c
        gcc $(CFLAGS) -o $@ antcalib.c

pdfs:
        echo $(ESFLAGS)
        enscript $(ESFLAGS) Makefile byteword.h debug.h escape.h token.h token.c test_token.c bcode.h bcode.c test_bcode.c program.h program.c test_prog.c container.h container.c function.h function.c eval.h eval.c test_eval.c test_assign.c basic.h basic.c pi_gpio.h pi_gpio.c main.c antcalib.c | ps2pdf $(PSFLAGS) - $(TARGET_DOC)/antbasic_src.pdf
        groff $(GROFFFLAGS) antbasic.1 | ps2pdf - $(TARGET_DOC)/antbasic_man.pdf

install: antbasic antcalib
        install -v -t $(TARGET_BIN) antbasic antcalib
        install -v -D -t $(TARGET_MAN) antbasic.1

clean:
        rm -f *.o $(APPS) $(TESTAPPS)

dist-clean:
        rm -f *.o $(APPS) $(TESTAPPS) $(PDFS)
        rm -f $(TARGET_BIN)/antbasic $(TARGET_BIN)/antcalib
        rm -f $(TARGET_MAN)/antbasic.1
```

```
//
//  b y t e w o r d . h
//
//  BAREMETALHACK.COM --> public domain
//

#ifndef _BYTEWORD_H_
#define _BYTEWORD_H_

#include <stdint.h>             // uint8_t, int16_t

//
// Type definitions
//

typedef uint8_t byte_t;
typedef int16_t word_t;

#endif  // _BYTEWORD_H_
```

```
//
//  d e b u g . h
//
//  BAREMETALHACK.COM --> public domain
//

#ifndef _DEBUG_H_
#define _DEBUG_H_

#include <stdio.h>      // FILE{}

//
// Global variable
//

extern FILE *Debug;

//
// Macro definition for debugging
//

#ifdef DEBUG
    #define DMSG(...) { \
        fprintf(Debug, "%s() ", __FUNCTION__); \
        fprintf(Debug, __VA_ARGS__); \
        fprintf(Debug, "\n"); \
    }
#else
    #define DMSG(...) {}
#endif

#endif // _DEBUG_H_
```

```
 1: //
 2: //  e s c a p e . h
 3: //
 4: //  BAREMETALHACK.COM --> public domain
 5: //
 6:
 7: #ifndef _ESCAPE_H_
 8: #define _ESCAPE_H_
 9:
10: //
11: // ANSI escape sequences
12: //
13:
14: #define ESCAPE_HOME             "\e[1;1H"
15: #define ESCAPE_CLS              "\e[2J"
16:
17: #define ESCAPE_DEFAULT          "\e[0m"
18: #define ESCAPE_BOLD             "\e[1m"
19: #define ESCAPE_RED              "\e[31m"
20: #define ESCAPE_GREEN            "\e[32m"
21: #define ESCAPE_YELLOW           "\e[33m"
22: #define ESCAPE_BLUE             "\e[34m"
23: #define ESCAPE_MAGENTA          "\e[35m"
24: #define ESCAPE_CYAN             "\e[36m"
25: #define ESCAPE_WHITE            "\e[37m"
26: #define ESCAPE_LRED             (ESCAPE_BOLD ESCAPE_RED)
27: #define ESCAPE_LGREEN           (ESCAPE_BOLD ESCAPE_GREEN)
28: #define ESCAPE_LYELLOW          (ESCAPE_BOLD ESCAPE_YELLOW)
29: #define ESCAPE_LBLUE            (ESCAPE_BOLD ESCAPE_BLUE)
30: #define ESCAPE_LMAGENTA         (ESCAPE_BOLD ESCAPE_MAGENTA)
31: #define ESCAPE_LCYAN            (ESCAPE_BOLD ESCAPE_CYAN)
32: #define ESCAPE_LWHITE           (ESCAPE_BOLD ESCAPE_WHITE)
33:
34: #endif  // _ESCAPE_H_
```

```
//
//  t o k e n . h
//
//  BAREMETALHACK.COM --> public domain
//

#ifndef _TOKEN_H_
#define _TOKEN_H_

#include "byteword.h"           // word_t

//
// Token constants
//

#define TOKEN_MAXBUF            512     // Source text buffer capacity
#define TOKEN_MAXSTR            256     // Maximum length of string
#define TOKEN_MAXKEY            10      // Maximum length of keyword
#define TOKEN_MAXDLM            2       // Maximum length of delimiter

//
// Token types
//

#define TOKEN_TYPE_EOL          0       // End Of Line
#define TOKEN_TYPE_NUMBER10     1       // Decimal number
#define TOKEN_TYPE_NUMBER16     2       // Hexa-decimal number
#define TOKEN_TYPE_STRING       3       // String
#define TOKEN_TYPE_VARIABLE     4       // Variable or array name
#define TOKEN_TYPE_DELIMITER    5       // Delimiter
#define TOKEN_TYPE_KEYWORD      6       // Keyword

//
// Index codes for variable
//

#define TOKEN_VAR_A     0
#define TOKEN_VAR_B     1
#define TOKEN_VAR_C     2
#define TOKEN_VAR_D     3
#define TOKEN_VAR_E     4
#define TOKEN_VAR_F     5
#define TOKEN_VAR_G     6
#define TOKEN_VAR_H     7
#define TOKEN_VAR_I     8
#define TOKEN_VAR_J     9
#define TOKEN_VAR_K     10
#define TOKEN_VAR_L     11
#define TOKEN_VAR_M     12
#define TOKEN_VAR_N     13
#define TOKEN_VAR_O     14
#define TOKEN_VAR_P     15
#define TOKEN_VAR_Q     16
#define TOKEN_VAR_R     17
#define TOKEN_VAR_S     18
#define TOKEN_VAR_T     19
#define TOKEN_VAR_U     20
#define TOKEN_VAR_V     21
#define TOKEN_VAR_W     22
#define TOKEN_VAR_X     23
#define TOKEN_VAR_Y     24
#define TOKEN_VAR_Z     25

//
// Index codes for delimiter
//

#define TOKEN_EQUAL     0       // ==
#define TOKEN_ASSIGN    1       // =
#define TOKEN_GEQUAL    2       // >=
#define TOKEN_GTHAN     3       // >
#define TOKEN_LEQUAL    4       // <=
#define TOKEN_LTHAN     5       // <
#define TOKEN_NEQUAL    6       // !=
#define TOKEN_AT        7       // @
#define TOKEN_LBRACK    8       // [
#define TOKEN_RBRACK    9       // ]
#define TOKEN_LPAREN    10      // (
#define TOKEN_RPAREN    11      // )
#define TOKEN_COLON     12      // :
#define TOKEN_SCOLON    13      // ;
#define TOKEN_COMMA     14      // ,
#define TOKEN_MUL       15      // *
#define TOKEN_DIV       16      // /
#define TOKEN_MOD       17      // %
#define TOKEN_ADD       18      // +
#define TOKEN_SUB       19      // -
#define TOKEN_AND       20      // &
#define TOKEN_OR        21      // |
#define TOKEN_INQ       22      // ?

//
// Index codes for reserved keyword
//

#define TOKEN_CLEAR     0
#define TOKEN_CLS       1
#define TOKEN_COLOR     2
#define TOKEN_DELETE    3
#define TOKEN_DIM       4
#define TOKEN_DUMP      5
#define TOKEN_EDIT      6
#define TOKEN_END       7
#define TOKEN_FOR       8
#define TOKEN_FREE      9
#define TOKEN_GOSUB     10
#define TOKEN_GOTO      11
#define TOKEN_HELP      12
#define TOKEN_HEX2      13
#define TOKEN_HEX4      14
#define TOKEN_IF        15
#define TOKEN_INPUT     16
#define TOKEN_LIST      17
#define TOKEN_LOCATE    18
#define TOKEN_NEW       19
#define TOKEN_NEXT      20
#define TOKEN_PRINT     21
#define TOKEN_REM       22
#define TOKEN_RENUM     23
#define TOKEN_RETURN    24
#define TOKEN_RND       25
#define TOKEN_RUN       26
#define TOKEN_TO        27

// --- OS dependent section ---

#define TOKEN_FILES     28
#define TOKEN_LOAD      29
#define TOKEN_MERGE     30
#define TOKEN_MSLEEP    31
#define TOKEN_SAVE      32
#define TOKEN_USLEEP    33
```

```
133:
134: #ifdef PI
135: #define TOKEN_IN        34
136: #define TOKEN_OUT       35
137: #define TOKEN_OUTHZ     36
138: #endif // PI
139:
140: //
141: // Result codes
142: //
143:
144: #define TOKEN_SUCCESS           0
145: #define TOKEN_MISSINGQUOTE      1
146: #define TOKEN_TOOLONGSTR        2
147: #define TOKEN_TOOLONGKEY        3
148: #define TOKEN_ILLEGALCHAR       4
149: #define TOKEN_ILLNUMBER         5
150: #define TOKEN_ILLSTRINRG        6
151: #define TOKEN_KEYNOTFOUND       7
152: #define TOKEN_DLMNOTFOUND       8
153: #define TOKEN_UNKNOWN           9
154:
155: //
156: // Type definition
157: //
158:
159: typedef struct {
160:     int type;
161:     union {
162:         int idx;
163:      word_t num;
164:        char str[ TOKEN_MAXSTR + 1 ];
165:     } body;
166: } token_t;
167:
168: //
169: // Function prototypes
170: //
171:
172:   void token_init(void);
173:   void token_source(char *);
174:    int token_read(token_t *);
175:  char *token_keyword(int);
176:  char *token_delimiter(int);
177:  char *token_result(int);
178:
179: #ifdef DEBUG
180: void token_dump(token_t *);
181: #endif // DEBUG
182:
183: #endif // _TOKEN_H_
```

```
//
//  t o k e n . c
//
//  BAREMETALHACK.COM --> public domain
//

#ifdef DEBUG            // token_dump()
#include <stdio.h>      // FILE{}, fprintf()
#include <string.h>     // strlen()
#endif

#include "token.h"
#include "debug.h"

//
// Constants for character type
//

#define CT_EOL              1       // end of line: NULL | '\n'
#define CT_SPACE            2       // ' ' | '\t'
#define CT_DELIMITER        4       // !()*+,-/:;<=>@[]&|?
#define CT_ALPHABET         8       // A-Z | a-z
#define CT_BINARY           16      // 0-1
#define CT_DECIMAL          32      // 0-9
#define CT_HEXADECIMAL      64      // 0-9 | A-F | a-f

//
// Local type definition
//

typedef struct {
    char * string;
       int index;
} table_t;

//
// File scope variables
//

static unsigned char Ctype[ 256 ];
static char *Srcptr;
static unsigned char Next;      // unsigned: safe as Ctype[] index for bytes >= 0x80

static table_t
Keywords[] = {
    { "CLEAR",  TOKEN_CLEAR },
    { "CLS",    TOKEN_CLS },
    { "COLOR",  TOKEN_COLOR },
    { "DELETE", TOKEN_DELETE },
    { "DIM",    TOKEN_DIM },
    { "DUMP",   TOKEN_DUMP },
    { "EDIT",   TOKEN_EDIT },
    { "END",    TOKEN_END },
    { "FOR",    TOKEN_FOR },
    { "FREE",   TOKEN_FREE },
    { "GOSUB",  TOKEN_GOSUB },
    { "GOTO",   TOKEN_GOTO },
    { "HELP",   TOKEN_HELP },
    { "HEX2",   TOKEN_HEX2 },
    { "HEX4",   TOKEN_HEX4 },
    { "IF",     TOKEN_IF },
    { "INPUT",  TOKEN_INPUT },
    { "LIST",   TOKEN_LIST },
    { "LOCATE", TOKEN_LOCATE },
    { "NEW",    TOKEN_NEW },
    { "NEXT",   TOKEN_NEXT },
    { "PRINT",  TOKEN_PRINT },
    { "REM",    TOKEN_REM },
    { "RENUM",  TOKEN_RENUM },
    { "RETURN", TOKEN_RETURN },
    { "RND",    TOKEN_RND },
    { "RUN",    TOKEN_RUN },
    { "TO",     TOKEN_TO },

    { "FILES",  TOKEN_FILES },
    { "LOAD",   TOKEN_LOAD },
    { "MERGE",  TOKEN_MERGE },
    { "MSLEEP", TOKEN_MSLEEP },
    { "SAVE",   TOKEN_SAVE },
    { "USLEEP", TOKEN_USLEEP },

#ifdef PI
    { "IN",     TOKEN_IN },
    { "OUT",    TOKEN_OUT },
    { "OUTHZ",  TOKEN_OUTHZ },
#endif // PI
    { 0, 0 }                        // table end mark
};

static table_t
Delimiters[] = {
    { "==", TOKEN_EQUAL },
    { "=",  TOKEN_ASSIGN },
    { ">=", TOKEN_GEQUAL },
    { ">",  TOKEN_GTHAN },
    { "<=", TOKEN_LEQUAL },
    { "<",  TOKEN_LTHAN },
    { "!=", TOKEN_NEQUAL },
    { "@",  TOKEN_AT },
    { "[",  TOKEN_LBRACK },
    { "]",  TOKEN_RBRACK },
    { "(",  TOKEN_LPAREN },
    { ")",  TOKEN_RPAREN },
    { ":",  TOKEN_COLON },
    { ";",  TOKEN_SCOLON },
    { ",",  TOKEN_COMMA },
    { "*",  TOKEN_MUL },
    { "/",  TOKEN_DIV },
    { "%",  TOKEN_MOD },
    { "+",  TOKEN_ADD },
    { "-",  TOKEN_SUB },
    { "&",  TOKEN_AND },
    { "|",  TOKEN_OR },
    { "?",  TOKEN_INQ },
    { 0, 0 }                        // table end mark
};

static char *
Results[] = {
    "TOKEN \"ok\"",                     // TOKEN_SUCCESS
    "TOKEN \"missing quotation\"",      // TOKEN_MISSINGQUOTE
    "TOKEN \"too long string\"",        // TOKEN_TOOLONGSTR
    "TOKEN \"too long keyword\"",       // TOKEN_TOOLONGKEY
    "TOKEN \"illegal character\"",      // TOKEN_ILLCHAR
    "TOKEN \"illegal number\"",         // TOKEN_ILLNUMBER
    "TOKEN \"illegal string\"",         // TOKEN_ILLSTRING
    "TOKEN \"keyword not found\"",      // TOKEN_KEYNOTFOUND
    "TOKEN \"delimiter not found\"",    // TOKEN_DLMNOTFOUND
    "TOKEN \"unknown token\""           // TOKEN_UNKNOWN
};

//
```

```
133: // Procedures and functions
134: //
135:
136: void
137: token_init(void)
138: {
139:     int i;
140:
141:     for (i = 0; i < 256; i++) Ctype[ i ] = 0;
142:
143:     Ctype[ 0 ] = Ctype[ '\n' ] = CT_EOL;
144:
145:     Ctype[ ' ' ] = Ctype[ '\t' ] = CT_SPACE;
146:
147:     Ctype[ '!' ] = Ctype[ '%' ] = Ctype[ '&' ] = Ctype[ '(' ] = \
148:     Ctype[ ')' ] = Ctype[ '*' ] = Ctype[ '+' ] = Ctype[ ',' ] = \
149:     Ctype[ '-' ] = Ctype[ '/' ] = Ctype[ ':' ] = Ctype[ ';' ] = \
150:     Ctype[ '<' ] = Ctype[ '=' ] = Ctype[ '>' ] = Ctype[ '@' ] = \
151:     Ctype[ '[' ] = Ctype[ ']' ] = Ctype[ '?' ] = Ctype[ '|' ] = CT_DELIMITER;
152:
153:     Ctype[ 'A' ] = Ctype[ 'B' ] = Ctype[ 'C' ] = Ctype[ 'D' ] = \
154:     Ctype[ 'E' ] = Ctype[ 'F' ] = Ctype[ 'G' ] = Ctype[ 'H' ] = \
155:     Ctype[ 'I' ] = Ctype[ 'J' ] = Ctype[ 'K' ] = Ctype[ 'L' ] = \
156:     Ctype[ 'M' ] = Ctype[ 'N' ] = Ctype[ 'O' ] = Ctype[ 'P' ] = \
157:     Ctype[ 'Q' ] = Ctype[ 'R' ] = Ctype[ 'S' ] = Ctype[ 'T' ] = \
158:     Ctype[ 'U' ] = Ctype[ 'V' ] = Ctype[ 'W' ] = Ctype[ 'X' ] = \
159:     Ctype[ 'Y' ] = Ctype[ 'Z' ] = CT_ALPHABET;
160:
161:     Ctype[ 'a' ] = Ctype[ 'b' ] = Ctype[ 'c' ] = Ctype[ 'd' ] = \
162:     Ctype[ 'e' ] = Ctype[ 'f' ] = Ctype[ 'g' ] = Ctype[ 'h' ] = \
163:     Ctype[ 'i' ] = Ctype[ 'j' ] = Ctype[ 'k' ] = Ctype[ 'l' ] = \
164:     Ctype[ 'm' ] = Ctype[ 'n' ] = Ctype[ 'o' ] = Ctype[ 'p' ] = \
165:     Ctype[ 'q' ] = Ctype[ 'r' ] = Ctype[ 's' ] = Ctype[ 't' ] = \
166:     Ctype[ 'u' ] = Ctype[ 'v' ] = Ctype[ 'w' ] = Ctype[ 'x' ] = \
167:     Ctype[ 'y' ] = Ctype[ 'z' ] = CT_ALPHABET;
168:
169:     Ctype[ '0' ] = Ctype[ '1' ] = Ctype[ '2' ] = Ctype[ '3' ] = \
170:     Ctype[ '4' ] = Ctype[ '5' ] = Ctype[ '6' ] = Ctype[ '7' ] = \
171:     Ctype[ '8' ] = Ctype[ '9' ] = CT_DECIMAL;
172:
173:     Ctype[ '0' ] |= Ctype[ '1' ] |= CT_BINARY;
174:
175:     Ctype[ '0' ] |= Ctype[ '1' ] |= Ctype[ '2' ] |= Ctype[ '3' ] |= \
176:     Ctype[ '4' ] |= Ctype[ '5' ] |= Ctype[ '6' ] |= Ctype[ '7' ] |= \
177:     Ctype[ '8' ] |= Ctype[ '9' ] |= CT_HEXADECIMAL;
178:
179:     Ctype[ 'A' ] |= Ctype[ 'B' ] |= Ctype[ 'C' ] |= Ctype[ 'D' ] |= \
180:     Ctype[ 'E' ] |= Ctype[ 'F' ] |= CT_HEXADECIMAL;
181:
182:     Ctype[ 'a' ] |= Ctype[ 'b' ] |= Ctype[ 'c' ] |= Ctype[ 'd' ] |= \
183:     Ctype[ 'e' ] |= Ctype[ 'f' ] |= CT_HEXADECIMAL;
184: }
185:
186: static void
187: read_char(void)
188: {
189:     Next = *Srcptr++;
190:     return;
191: }
192:
193: void
194: token_source(char *ptr)
195: {
196:     Srcptr = ptr;
197:     read_char();        // do pre-fetch first
198:     return;
199: }
200:
201: static int
202: check_digit(int base)
203: {
204:     if (base == 16) {
205:         if (Ctype[ (int) Next ] & CT_HEXADECIMAL) {
206:             if (Next >= 'a')
207:                 return (Next - 'a' + 10);
208:             else if (Next >= 'A')
209:                 return (Next - 'A' + 10);
210:             else
211:                 return (Next - '0');
212:         } else
213:             return -1;
214:     } else if (base == 10) {
215:         if (Ctype[ (int) Next ] & CT_DECIMAL)
216:             return (Next - '0');
217:         else
218:             return -1;
219:     } else if (base == 2) {
220:         if (Ctype[ (int) Next ] & CT_BINARY)
221:             return (Next - '0');
222:         else
223:             return -1;
224:     } else
225:         return -1;
226: }
227:
228: static word_t
229: number(token_t *tok)
230: {
231:     word_t base = 10, digit, val = 0;
232:
233:     if (Next == '0') {
234:         read_char();
235:         if (Next == 'x' || Next == 'X') {
236:             base = 16;
237:             read_char();
238:         } else
239:             base = 10;
240:     }
241:     while ((digit = check_digit(base)) >= 0) {
242:         val = (val * base) + digit;
243:         read_char();
244:     };
245:
246:     if (base == 16)
247:         tok->type = TOKEN_TYPE_NUMBER16;
248:     else
249:         tok->type = TOKEN_TYPE_NUMBER10;
250:     tok->body.num = val;
251:     return TOKEN_SUCCESS;
252: }
253:
254: static int
255: string(token_t *tok)
256: {
257:     int cnt, high, low;
258:     char *str;
259:
260:     str = tok->body.str;
261:     cnt = 0;
262:     read_char();
263:     while (Next && Next != '\"' && cnt < TOKEN_MAXSTR) {
264:         if (Next == '\\') {
```

```
            read_char();
            switch(Next) {
                case 'a':
                    Next = '\007';
                    break;

                case 'b':
                    Next = '\010';
                    break;

                case 't':
                    Next = '\011';
                    break;

                case 'n':
                    Next = '\012';
                    break;

                case 'r':
                    Next = '\015';
                    break;

                case 'e':
                    Next = '\033';
                    break;

                case 'x':
                    read_char();
                    if (! (Ctype[ (int) Next ] & CT_HEXADECIMAL))
                        return TOKEN_ILLNUMBER;
                    high = check_digit(16);
                    read_char();
                    if (! (Ctype[ (int) Next ] & CT_HEXADECIMAL))
                        return TOKEN_ILLNUMBER;
                    low = check_digit(16);
                    Next = high * 16 + low;;
                    break;

                case '\\':
                    Next = '\\';
                    break;

                default:                // unknown character
                    Next = '?';
                    break;
            }
        }
        *str++ = Next;
        cnt++;
        read_char();
    }
    *str = 0;

    if (Next == '\"')
        read_char();
    else if (cnt == TOKEN_MAXSTR)
        return TOKEN_TOOLONGSTR;
    else
        return TOKEN_MISSINGQUOTE;

    tok->type = TOKEN_TYPE_STRING;
    return TOKEN_SUCCESS;
}

static int
delimiter(token_t *tok)
{
    int idx = 0;                // never left at 0: token_read() calls us only for CT_DELIMITER

    switch(Next) {
        case '=':
            read_char();
            if (Next == '=') {
                read_char();
                idx = TOKEN_EQUAL;
            } else
                idx = TOKEN_ASSIGN;
            break;

        case '>':
            read_char();
            if (Next == '=') {
                read_char();
                idx = TOKEN_GEQUAL;
            } else
                idx = TOKEN_GTHAN;
            break;

        case '<':
            read_char();
            if (Next == '=') {
                read_char();
                idx = TOKEN_LEQUAL;
            } else
                idx = TOKEN_LTHAN;
            break;

        case '!':
            read_char();
            if (Next == '=') {
                read_char();
                idx = TOKEN_NEQUAL;
            } else
                return TOKEN_UNKNOWN;
            break;

        case '@':
            read_char();
            idx = TOKEN_AT;
            break;

        case '[':
            read_char();
            idx = TOKEN_LBRACK;
            break;

        case ']':
            read_char();
            idx = TOKEN_RBRACK;
            break;

        case '(':
            read_char();
            idx = TOKEN_LPAREN;
            break;

        case ')':
            read_char();
            idx = TOKEN_RPAREN;
            break;

```

```
        case ':':
            read_char();
            idx = TOKEN_COLON;
            break;

        case ';':
            read_char();
            idx = TOKEN_SCOLON;
            break;

        case ',':
            read_char();
            idx = TOKEN_COMMA;
            break;

        case '*':
            read_char();
            idx = TOKEN_MUL;
            break;

        case '/':
            read_char();
            idx = TOKEN_DIV;
            break;

        case '%':
            read_char();
            idx = TOKEN_MOD;
            break;

        case '+':
            read_char();
            idx = TOKEN_ADD;
            break;

        case '-':
            read_char();
            idx = TOKEN_SUB;
            break;
        case '&':
            read_char();
            idx = TOKEN_AND;
            break;

        case '|':
            read_char();
            idx = TOKEN_OR;
            break;

        case '?':
            read_char();
            idx = TOKEN_INQ;
            break;
    }
    tok->type = TOKEN_TYPE_DELIMITER;
    tok->body.idx = idx;
    return TOKEN_SUCCESS;
}

static int
search_table(table_t *tbl, char *key)
{
    int idx = 0;
    char *src, *dst;

    while (tbl->string) {
        src = key;
        dst = tbl->string;
        while (*src && *dst) {
            if (*dst != *src)
                break;
            src++;
            dst++;
        }
        if ((*src == 0) && (*dst == 0))
            return idx;
        idx++;
        tbl++;
    }
    return -1;
}

static int
keyword(token_t *tok)
{
    int cnt, idx;
    char key[ TOKEN_MAXKEY + 1 ], *ptr;

    ptr = key;
    cnt = 0;
    while (Ctype[ (int) Next ] & CT_ALPHABET || \
           Ctype[ (int) Next ] & CT_DECIMAL) {
        if (Next >= 'a' && Next <= 'z')
            Next -= 0x20;
        cnt++;
        if (cnt == (TOKEN_MAXKEY + 1))
            return TOKEN_TOOLONGKEY;
        *ptr++ = Next;
        read_char();
    }
    *ptr = 0;

    if (cnt == 1) {
        tok->type = TOKEN_TYPE_VARIABLE;
        tok->body.idx = key[ 0 ] - 'A';
        return TOKEN_SUCCESS;
    }

    idx = search_table(Keywords, key);
    if (idx < 0)
        return TOKEN_KEYNOTFOUND;

    tok->type = TOKEN_TYPE_KEYWORD;
    tok->body.idx = Keywords[ idx ].index;
    return TOKEN_SUCCESS;
}

int
token_read(token_t *tok)
{
    while (Ctype[ (int) Next ] & CT_SPACE)      // Skip blank
        read_char();
    if (Ctype[ (int) Next ] & CT_EOL) {
        tok->type = TOKEN_TYPE_EOL;
        return TOKEN_SUCCESS;
    }

    if (Ctype[ (int) Next ] & CT_DECIMAL)
        return number(tok);
    else if (Ctype[ (int) Next ] & CT_ALPHABET)
        return keyword(tok);
    else if (Ctype[ (int) Next ] & CT_DELIMITER)
```

```
528:         return delimiter(tok);
529:     else if (Next == '\"')
530:         return string(tok);
531:     else
532:         return TOKEN_UNKNOWN;
533: }
534:
535: char *
536: token_keyword(int idx)
537: {
538:     return Keywords[ idx ].string;
539: }
540:
541: char *
542: token_delimiter(int idx)
543: {
544:     return Delimiters[ idx ].string;
545: }
546:
547: char *
548: token_result(int idx)
549: {
550:     return Results[ idx ];
551: }
552:
553: #ifdef DEBUG
554: void
555: token_dump(token_t *tok)
556: {
557:     int i, len;
558:
559:     fprintf(Debug, "    token type %d : ", tok->type);
560:     switch (tok->type) {
561:         case TOKEN_TYPE_EOL:
562:             fprintf(Debug, "EOL\n");
563:             break;
564:
565:         case TOKEN_TYPE_NUMBER10:
566:             fprintf(Debug, "number %d (0x%0hX)\n", tok->body.num, tok->body.num);
567:             break;
568:
569:         case TOKEN_TYPE_NUMBER16:
570:             fprintf(Debug, "number 0x%0hX (%d)\n", tok->body.num, tok->body.num);
571:             break;
572:
573:         case TOKEN_TYPE_STRING:
574:             len = strlen(tok->body.str);
575:             fprintf(Debug, "string (strlen = %d)\n", len);
576:             for (i = 0; i < (len + 1); i++) {
577:                 if ((i % 16) == 0)
578:                     fprintf(Debug, "        ");
579:                 fprintf(Debug, "%02hhX ", (unsigned char) tok->body.str[ i ]);
580:                 if ((i % 16) == 15)
581:                     fprintf(Debug, "\n");
582:             }
583:             if ((i % 16) != 15)
584:                 fprintf(Debug, "\n");
585:             break;
586:
587:         case TOKEN_TYPE_VARIABLE:
588:             fprintf(Debug, "variable %c\n", tok->body.idx + 'A');
589:             break;
590:
591:         case TOKEN_TYPE_KEYWORD:
592:             fprintf(Debug, "keyword \"%s\"\n", Keywords[ tok->body.idx ].string);
593:             break;
594:
595:         case TOKEN_TYPE_DELIMITER:
596:             fprintf(Debug, "delimiter \"%s\"\n", Delimiters[ tok->body.idx ].string)
;
597:             break;
598:     }
599: }
600: #endif // DEBUG
```

```
 1: #include <stdio.h>      // FILE{}, stdin, stderr, fgets(), printf()
 2: #include "token.h"
 3:
 4: FILE *Debug;
 5:
 6: int
 7: main()
 8: {
 9:     char buffer[ 256 ];
10:     token_t tok;
11:     int ret;
12:
13:     Debug = stderr;
14:     token_init();
15:     while (fgets(buffer, 256, stdin) != NULL) {
16:         if (*buffer == 0 || *buffer == '\n')
17:             break;
18:         printf("===== %s", buffer);
19:         token_source(buffer);
20:         while (1) {
21:             ret = token_read(&tok);
22:             if (ret != TOKEN_SUCCESS) {
23:                 fprintf(Debug, "*** Token error: %s\n", token_result(ret));
24:                 break;
25:             }
26: #ifdef DEBUG
27:             token_dump(&tok);
28: #endif
29:             if (tok.type == TOKEN_TYPE_EOL)
30:                 break;
31:         }
32:         fprintf(Debug, "\n");
33:     }
34:
35:     return 0;
36: }
```

```
//
//  b c o d e . h
//
//  BAREMETALHACK.COM --> public domain
//

#ifndef _BCODE_H_
#define _BCODE_H_

#include "token.h"

//
// Bytecode constants
//

#define BCODE_MAXSIZE           512     // Maximum byte size in single line

//-------------------------------------------------
//
// Bytecode instruction codes
//
//   Type 0       EOL: 0x00
//   Type 1  Number10: 0x01, low, high
//   Type 2  Number16: 0x02, low, high
//   Type 3    String: 0x03, len (includes null terminator), ch1, ch2, ..., 0
//   Type 4  Variable: 0x10 - 0x29
//   Type 5 Delimiter: 0x40 - 0x56
//   Type 6   Keyword: 0x80 - 0xA3
//
//-------------------------------------------------

#define BCODE_TYPE_EOL          TOKEN_TYPE_EOL
#define BCODE_TYPE_NUMBER10     TOKEN_TYPE_NUMBER10
#define BCODE_TYPE_NUMBER16     TOKEN_TYPE_NUMBER16
#define BCODE_TYPE_STRING       TOKEN_TYPE_STRING
#define BCODE_TYPE_VARIABLE     TOKEN_TYPE_VARIABLE
#define BCODE_TYPE_DELIMITER    TOKEN_TYPE_DELIMITER
#define BCODE_TYPE_KEYWORD      TOKEN_TYPE_KEYWORD

#define BCODE_EOL               BCODE_TYPE_EOL
#define BCODE_NUMBER10          BCODE_TYPE_NUMBER10
#define BCODE_NUMBER16          BCODE_TYPE_NUMBER16
#define BCODE_STRING            BCODE_TYPE_STRING
#define BCODE_VARIABLE          0x10
#define BCODE_DELIMITER         0x40
#define BCODE_KEYWORD           0x80

#define BCODE_VAR_A             (BCODE_VAIRABLE | ('A' - 'A'))
#define BCODE_VAR_B             (BCODE_VAIRABLE | ('B' - 'A'))
#define BCODE_VAR_C             (BCODE_VAIRABLE | ('C' - 'A'))
#define BCODE_VAR_D             (BCODE_VAIRABLE | ('D' - 'A'))
#define BCODE_VAR_E             (BCODE_VAIRABLE | ('E' - 'A'))
#define BCODE_VAR_F             (BCODE_VAIRABLE | ('F' - 'A'))
#define BCODE_VAR_G             (BCODE_VAIRABLE | ('G' - 'A'))
#define BCODE_VAR_H             (BCODE_VAIRABLE | ('H' - 'A'))
#define BCODE_VAR_I             (BCODE_VAIRABLE | ('I' - 'A'))
#define BCODE_VAR_J             (BCODE_VAIRABLE | ('J' - 'A'))
#define BCODE_VAR_K             (BCODE_VAIRABLE | ('K' - 'A'))
#define BCODE_VAR_L             (BCODE_VAIRABLE | ('L' - 'A'))
#define BCODE_VAR_M             (BCODE_VAIRABLE | ('M' - 'A'))
#define BCODE_VAR_N             (BCODE_VAIRABLE | ('N' - 'A'))
#define BCODE_VAR_O             (BCODE_VAIRABLE | ('O' - 'A'))
#define BCODE_VAR_P             (BCODE_VAIRABLE | ('P' - 'A'))
#define BCODE_VAR_Q             (BCODE_VAIRABLE | ('Q' - 'A'))
#define BCODE_VAR_R             (BCODE_VAIRABLE | ('R' - 'A'))
#define BCODE_VAR_S             (BCODE_VAIRABLE | ('S' - 'A'))
#define BCODE_VAR_T             (BCODE_VAIRABLE | ('T' - 'A'))
#define BCODE_VAR_U             (BCODE_VAIRABLE | ('U' - 'A'))
#define BCODE_VAR_V             (BCODE_VAIRABLE | ('V' - 'A'))
#define BCODE_VAR_W             (BCODE_VAIRABLE | ('W' - 'A'))
#define BCODE_VAR_X             (BCODE_VAIRABLE | ('X' - 'A'))
#define BCODE_VAR_Y             (BCODE_VAIRABLE | ('Y' - 'A'))
#define BCODE_VAR_Z             (BCODE_VAIRABLE | ('Z' - 'A'))

#define BCODE_EQUAL             (BCODE_DELIMITER | TOKEN_EQUAL)
#define BCODE_ASSIGN            (BCODE_DELIMITER | TOKEN_ASSIGN)
#define BCODE_GEQUAL            (BCODE_DELIMITER | TOKEN_GEQUAL)
#define BCODE_GTHAN             (BCODE_DELIMITER | TOKEN_GTHAN)
#define BCODE_LEQUAL            (BCODE_DELIMITER | TOKEN_LEQUAL)
#define BCODE_LTHAN             (BCODE_DELIMITER | TOKEN_LTHAN)
#define BCODE_NEQUAL            (BCODE_DELIMITER | TOKEN_NEQUAL)
#define BCODE_EXCL              (BCODE_DELIMITER | TOKEN_EXCL)
#define BCODE_AT                (BCODE_DELIMITER | TOKEN_AT)
#define BCODE_LBRACK            (BCODE_DELIMITER | TOKEN_LBRACK)
#define BCODE_RBRACK            (BCODE_DELIMITER | TOKEN_RBRACK)
#define BCODE_LPAREN            (BCODE_DELIMITER | TOKEN_LPAREN)
#define BCODE_RPAREN            (BCODE_DELIMITER | TOKEN_RPAREN)
#define BCODE_COLON             (BCODE_DELIMITER | TOKEN_COLON)
#define BCODE_SCOLON            (BCODE_DELIMITER | TOKEN_SCOLON)
#define BCODE_COMMA             (BCODE_DELIMITER | TOKEN_COMMA)
#define BCODE_MUL               (BCODE_DELIMITER | TOKEN_MUL)
#define BCODE_DIV               (BCODE_DELIMITER | TOKEN_DIV)
#define BCODE_MOD               (BCODE_DELIMITER | TOKEN_MOD)
#define BCODE_ADD               (BCODE_DELIMITER | TOKEN_ADD)
#define BCODE_SUB               (BCODE_DELIMITER | TOKEN_SUB)
#define BCODE_AND               (BCODE_DELIMITER | TOKEN_AND)
#define BCODE_OR                (BCODE_DELIMITER | TOKEN_OR)
#define BCODE_INQ               (BCODE_DELIMITER | TOKEN_INQ)

#define BCODE_CLEAR             (BCODE_KEYWORD | TOKEN_CLEAR)
#define BCODE_CLS               (BCODE_KEYWORD | TOKEN_CLS)
#define BCODE_COLOR             (BCODE_KEYWORD | TOKEN_COLOR)
#define BCODE_DELETE            (BCODE_KEYWORD | TOKEN_DELETE)
#define BCODE_DIM               (BCODE_KEYWORD | TOKEN_DIM)
#define BCODE_DUMP              (BCODE_KEYWORD | TOKEN_DUMP)
#define BCODE_EDIT              (BCODE_KEYWORD | TOKEN_EDIT)
#define BCODE_END               (BCODE_KEYWORD | TOKEN_END)
#define BCODE_FOR               (BCODE_KEYWORD | TOKEN_FOR)
#define BCODE_FREE              (BCODE_KEYWORD | TOKEN_FREE)
#define BCODE_GOSUB             (BCODE_KEYWORD | TOKEN_GOSUB)
#define BCODE_GOTO              (BCODE_KEYWORD | TOKEN_GOTO)
#define BCODE_HELP              (BCODE_KEYWORD | TOKEN_HELP)
#define BCODE_HEX2              (BCODE_KEYWORD | TOKEN_HEX2)
#define BCODE_HEX4              (BCODE_KEYWORD | TOKEN_HEX4)
#define BCODE_IF                (BCODE_KEYWORD | TOKEN_IF)
#define BCODE_INPUT             (BCODE_KEYWORD | TOKEN_INPUT)
#define BCODE_LIST              (BCODE_KEYWORD | TOKEN_LIST)
#define BCODE_LOCATE            (BCODE_KEYWORD | TOKEN_LOCATE)
#define BCODE_NEW               (BCODE_KEYWORD | TOKEN_NEW)
#define BCODE_NEXT              (BCODE_KEYWORD | TOKEN_NEXT)
#define BCODE_PRINT             (BCODE_KEYWORD | TOKEN_PRINT)
#define BCODE_REM               (BCODE_KEYWORD | TOKEN_REM)
#define BCODE_RENUM             (BCODE_KEYWORD | TOKEN_RENUM)
#define BCODE_RETURN            (BCODE_KEYWORD | TOKEN_RETURN)
#define BCODE_RND               (BCODE_KEYWORD | TOKEN_RND)
#define BCODE_RUN               (BCODE_KEYWORD | TOKEN_RUN)
#define BCODE_TO                (BCODE_KEYWORD | TOKEN_TO)

#define BCODE_FILES             (BCODE_KEYWORD | TOKEN_FILES)
#define BCODE_LOAD              (BCODE_KEYWORD | TOKEN_LOAD)
#define BCODE_MERGE             (BCODE_KEYWORD | TOKEN_MERGE)
#define BCODE_MSLEEP            (BCODE_KEYWORD | TOKEN_MSLEEP)
```

```
133: #define BCODE_SAVE              (BCODE_KEYWORD | TOKEN_SAVE)
134: #define BCODE_USLEEP            (BCODE_KEYWORD | TOKEN_USLEEP)
135:
136: #ifdef PI
137: #define BCODE_IN                (BCODE_KEYWORD | TOKEN_IN)
138: #define BCODE_OUT               (BCODE_KEYWORD | TOKEN_OUT)
139: #define BCODE_OUTHZ             (BCODE_KEYWORD | TOKEN_OUTHZ)
140: #endif // PI
141:
142: //
143: // Result codes
144: //
145:
146: #define BCODE_SUCCESS           TOKEN_SUCCESS
147: #define BOCDE_MISSINGQUOTE      TOKEN_MISSINGQUOTE
148: #define BCODE_TOOLONGSTR        TOKEN_TOOLONGSTR
149: #define BCODE_TOOLONGKEY        TOKEN_TOOLONGKEY
150: #define BCODE_ILLEGALCHAR       TOKEN_ILLEGALCHAR
151: #define BCODE_ILLNUMBER         TOKEN_ILLNUMBER
152: #define BCODE_ILLSTRINRG        TOKEN_ILLSTRING
153: #define BCODE_KEYNOTFOUND       TOKEN_KEYNOTFOUND
154: #define BCODE_DLMNOTFOUND       TOKEN_DLMNOTFOUND
155: #define BCODE_UNKNOWN           TOKEN_UNKNOWN
156: #define BCODE_SIZEOVER          (BCODE_UNKNOWN + 1)
157:
158: //
159: // Type definition
160: //
161:
162: typedef struct {
163:       int pos;
164:       int inst;
165:       int type;
166:       int idx;
167:    word_t num;
168:  byte_t * str;
169: } bcode_t;
170:
171: //
172: // Function prototypes
173: //
174:
175:    void bcode_start(byte_t *);
176:    void bcode_setpc(int);
177:     int bcode_getpc(void);
178:    void bcode_skip(void);
179:     int bcode_next(void);
180:     int bcode_nextiseol(void);
181:     int bcode_nextisnum(void);
182:     int bcode_nextisstr(void);
183:     int bcode_nextisvar(void);
184:     int bcode_nextiskey(void);
185:     int bcode_nextisdlm(void);
186:     int bcode_twoahead(void);
187:    void bcode_read(bcode_t *);
188:     int bcode_compile(byte_t *);
189:   char *bcode_result(int);
190:
191: #ifdef DEBUG
192:    void bcode_display(bcode_t *);
193:    void bcode_dump(byte_t *);
194: #endif // DEBUG
195:
196: #endif // _BCODE_H_
```

```
//
//  b c o d e . c
//
//  BAREMETALHACK.COM --> public domain
//

#ifdef DEBUG                // bcode_dump()
#include <stdio.h>          // FILE{}, fprintf()
#endif

#include "bcode.h"

//
// Filescope variables
//

static byte_t *Code;        // points start of code area
static int Pc;              // program counter

//
// Bytecode functions
//

void
bcode_start(byte_t *ptr)
{
    Code = ptr;
    Pc = 0;
}

void
bcode_setpc(int where)
{
    Pc = where;
}

int
bcode_getpc(void)
{
    return Pc;
}

void
bcode_skip(void)
{
    bcode_t b;

    bcode_read(&b);
}

int
bcode_next(void)
{
    return (int) Code[ Pc ];
}

int
bcode_nextiseol(void)
{
    return Code[ Pc ] == BCODE_EOL ? 1 : 0;
}

int
bcode_nextisnum(void)
{
    if ((Code[ Pc ] == BCODE_NUMBER10) || (Code[ Pc ] == BCODE_NUMBER16))
        return 1;
    else
        return 0;
}

int
bcode_nextisstr(void)
{
    return Code[ Pc ] == BCODE_STRING ? 1 : 0;
}

int
bcode_nextisvar(void)
{
    if ((Code[ Pc ] >= BCODE_VARIABLE) && (Code[ Pc ] < BCODE_DELIMITER))
        return 1;
    else
        return 0;
}

int
bcode_nextisdlm(void)
{
    if ((Code[ Pc ] >= BCODE_DELIMITER) && (Code[ Pc ] < BCODE_KEYWORD))
        return 1;
    else
        return 0;
}

int
bcode_nextiskey(void)
{
    return Code[ Pc ] >= BCODE_KEYWORD ? 1 : 0;
}

int
bcode_twoahead(void)
{
    return (int) Code[ Pc + 1 ];
}

void
bcode_read(bcode_t *b)
{
    int len;

    b->pos = Pc;
    b->inst = Code[ Pc ];
    if (Code[ Pc ] < BCODE_VARIABLE)
        b->type = b->idx = Code[ Pc ];
    else if (Code[ Pc ] >= BCODE_VARIABLE && Code[ Pc ] < BCODE_DELIMITER) {
        b->type = BCODE_TYPE_VARIABLE;
        b->idx = Code[ Pc ] - BCODE_VARIABLE;
    } else if (Code[ Pc ] >= BCODE_DELIMITER && Code[ Pc ] < BCODE_KEYWORD) {
        b->type = BCODE_TYPE_DELIMITER;
        b->idx = Code[ Pc ] - BCODE_DELIMITER;
    } else {
        b->type = BCODE_TYPE_KEYWORD;
        b->idx = Code[ Pc ] - BCODE_KEYWORD;
    }
    b->num = 0;
    b->str = 0;
    Pc++;
    switch (b->type) {
        case BCODE_TYPE_NUMBER10:
        case BCODE_TYPE_NUMBER16:
```

```
  133:             b->num = (word_t) (Code[ Pc ] | (Code[ Pc + 1 ] << 8));  // little endia
n, no alignment assumption
  134:             Pc += 2;
  135:             break;
  136:
  137:         case BCODE_TYPE_STRING:
  138:             b->idx = len = Code[ Pc++ ];
  139:             b->str = &Code[ Pc ];
  140:             Pc += len;
  141:             break;
  142:     }
  143: }
  144:
  145: int
  146: bcode_compile(byte_t *start)
  147: {
  148:     byte_t *bptr, *lptr, *sptr;
  149:     int len, ret;
  150:     token_t tok;
  151:
  152:     bptr = start;
  153:     while (1) {
  154:         ret = token_read(&tok);
  155:         if (ret != TOKEN_SUCCESS)
  156:             return -ret;
  157:         switch (tok.type) {
  158:             case TOKEN_TYPE_EOL:
  159:                 *bptr++ = BCODE_EOL;
  160:                 return bptr - start;
  161:
  162:             case TOKEN_TYPE_NUMBER10:
  163:                 *bptr++ = BCODE_NUMBER10;
  164:                 *bptr++ = tok.body.num & 0xFF;
  165:                 *bptr++ = (tok.body.num >> 8) & 0xFF;
  166:                 break;
  167:
  168:             case TOKEN_TYPE_NUMBER16:
  169:                 *bptr++ = BCODE_NUMBER16;
  170:                 *bptr++ = tok.body.num & 0xFF;
  171:                 *bptr++ = (tok.body.num >> 8) & 0xFF;
  172:                 break;
  173:
  174:             case TOKEN_TYPE_STRING:
  175:                 *bptr++ = BCODE_STRING;
  176:                 lptr = bptr++;
  177:                 sptr = (byte_t *) tok.body.str;
  178:                 len = 0;
  179:                 while (*sptr) {
  180:                     *bptr++ = *sptr++;
  181:                     len++;
  182:                 }
  183:                 *bptr++ = 0;
  184:                 len++;
  185:                 *lptr = len;
  186:                 break;
  187:
  188:             case TOKEN_TYPE_VARIABLE:
  189:                 *bptr++ = BCODE_VARIABLE + tok.body.idx;
  190:                 break;
  191:
  192:             case TOKEN_TYPE_DELIMITER:
  193:                 *bptr++ = BCODE_DELIMITER + tok.body.idx;
  194:                 break;
  195:
  196:             case TOKEN_TYPE_KEYWORD:
  197:                 *bptr++ = BCODE_KEYWORD + tok.body.idx;
  198:                 break;
  199:         }
  200:         if ((bptr - start + 3) > BCODE_MAXSIZE)
  201:             return -BCODE_SIZEOVER;
  202:     }
  203:     return bptr - start;
  204: }
  205:
  206: char *
  207: bcode_result(int idx)
  208: {
  209:     static char
  210:         msg_ok[] = "BCODE \"ok\"",
  211:         msg_sizeover[] = "BCODE \"size over\"";
  212:
  213:     if (idx == BCODE_SUCCESS)
  214:         return msg_ok;
  215:     else if (idx == BCODE_SIZEOVER)
  216:         return msg_sizeover;
  217:     else
  218:         return token_result(idx);
  219: }
  220:
  221: #ifdef DEBUG
  222: extern FILE *Debug;
  223:
  224: void
  225: bcode_display(bcode_t *b)
  226: {
  227:     fprintf(Debug, "    0d%05d : ", b->pos);
  228:     fprintf(Debug, "bcode 0x%02X : ", b->inst);
  229:     fprintf(Debug, "type %1d : ", b->type);
  230:     fprintf(Debug, "idx %2d | ", b->idx);
  231:     if (b->type == BCODE_TYPE_NUMBER10)
  232:         fprintf(Debug, "%d (0x%04hX)\n", b->num, b->num);
  233:     else if (b->type == BCODE_TYPE_NUMBER16)
  234:         fprintf(Debug, "0x%04hX (%d)\n", b->num, b->num);
  235:     else if (b->type == BCODE_TYPE_STRING)
  236:         fprintf(Debug, "\"%s\"\n", (char *) b->str);
  237:     else if (b->type == BCODE_TYPE_VARIABLE)
  238:         fprintf(Debug, "%c\n", 'A' + b->idx);
  239:     else if (b->type == BCODE_TYPE_KEYWORD)
  240:         fprintf(Debug, "%s\n", token_keyword(b->idx));
  241:     else if (b->type == BCODE_TYPE_DELIMITER)
  242:         fprintf(Debug, "%s\n", token_delimiter(b->idx));
  243:     else if (b->type == BCODE_TYPE_EOL)
  244:         fprintf(Debug, "EOL\n");
  245: }
  246:
  247: void
  248: bcode_dump(byte_t *bytes)
  249: {
  250:     bcode_t b;
  251:
  252:     fprintf(Debug, "[Compiled bytecodes]\n");
  253:     bcode_start(bytes);
  254:     while (1) {
  255:         bcode_read(&b);
  256:         bcode_display(&b);
  257:         if (b.type == BCODE_TYPE_EOL)
  258:             break;
  259:     }
  260: }
  261: #endif
```

```
#include <stdio.h>      // FILE{}, fgets()
#include "bcode.h"

FILE *Debug;

int
main()
{
    char buffer[ 256 ];
    byte_t bytes[ 256 ];
    int i, ret;

    Debug = stderr;
    token_init();
    while (fgets(buffer, 256, stdin) != NULL) {
        if (*buffer == 0 || *buffer == '\n')
            break;
        fprintf(Debug, "===== %s", buffer);
        token_source(buffer);
        ret = bcode_compile(bytes);
        if (ret < 0) {
            ret = -ret;
            fprintf(Debug, "***** Error: %s\n", bcode_result(ret));
            break;
        }
        fprintf(Debug, ">>>>> Compiled bytecodes [ %d bytes ]\n", ret);
        for (i = 0; i < ret; i++) {
            if ((i % 16) == 0)
                fprintf(Debug, "    ");
            fprintf(Debug, "%02X ", bytes[ i ]);
            if ((i % 16) == 15)
                fprintf(Debug, "\n");
        }
        if ((i % 16) != 15)
            fprintf(Debug, "\n");
#ifdef DEBUG
            bcode_dump(bytes);
#endif // DEBUG
    }
    return ret;
}
```

```
//
//  p r o g r a m . h
//
//  BAREMETALHACK.COM --> public domain
//

#ifndef _PROGRAM_H_
#define _PROGRAM_H_

#include <stdio.h>              // FILE{}
#include "bcode.h"

//
// Program constants
//

#define PROG_MAXSIZE        30000
#define PROG_MAXLINE        2000
#define PROG_MAXRELOC       100
#define PROG_BLANKLINE      0x7FFF
#define PROG_BLANKDATA      0xFF
#define PROG_CMDLINE        PROG_MAXSIZE

#define PROG_LISTCOLOR      1
#define PROG_LISTPLAIN      0

//
// Result codes
//

#define PROG_SUCCESS        0
#define PROG_LINENOTFOUND   1
#define PROG_TOOMANYLINES   2
#define PROG_SIZEOVER       3
#define PROG_TOOMANYRELOCS  4

//
// Type definitions
//

typedef struct {
    int num;
    int add;
    int len;
} line_t;

typedef struct {
    int idx;
    int add;
} reloc_t;

//
// Global variables
//

extern byte_t Program[];
extern int Psize;
extern line_t Lines[];
extern int Lsize;
extern int Pnum;

//
// Function prototypes
//

  void prog_init(void);
   int prog_list(FILE *, int, int, int);
   int prog_search(int);
   int prog_delete(int, int);
   int prog_insert(int, byte_t *, int);
  void prog_cmdline(byte_t *, int);
   int prog_renum(int, int);
 char *prog_result(int);
  void prog_dumpbytes(void);
  void prog_dumplines(void);

#endif // _PROGRAM_H_
```

```
//
//  p r o g r a m . c
//
//  BAREMETALHACK.COM --> public domain
//

#include <stdio.h>          // FILE{}, fprintf()
#include "program.h"
#include "bcode.h"
#include "escape.h"

//
// Global variables
//

  byte_t Program[ PROG_MAXSIZE + BCODE_MAXSIZE ]; // Last region (BCODE_MAXSIZE)
                                                  //     stores command line data
  line_t Lines[ PROG_MAXLINE + 2 ];     // First element is COMMAND LINE which
                                        //      contains ZERO
                                        // Last element is a GUARD which contains
                                        //      PROG_BLANKLINE
     int Psize;                         // Program size in BYTES
     int Lsize;                         // How many LINES are there?
     int Pnum;                          // Current processing line number

//
// File scope variables
//

static reloc_t Relocs[ PROG_MAXRELOC ];
    static int Rsize;

static char * Results[] = {
    "PROGRAM \"ok\"",                   // PROG_SUCCESS
    "PROGRAM \"line not found\"",       // PROG_LINENOTFOUND
    "PROGRAM \"too many lines\"",       // PROG_TOOMANYLINES
    "PROGRAM \"size over\"",            // PROG_SIZEOVER
    "PROGRAM \"too many relocations\""  // PROG_TOOMANYRELOCS
};

//
// Program line processing functions
//

void
prog_init(void)
{
    int i;

    // Initialize program area and line holders

    for (i = 0; i < (PROG_MAXSIZE + BCODE_MAXSIZE); i++)
        Program[ i ] = PROG_BLANKDATA;

    for (i = 0; i <= (PROG_MAXLINE + 1); i++) {
        Lines[ i ].num = PROG_BLANKLINE;
        Lines[ i ].add = 0;
        Lines[ i ].len = 0;
    }
    Lines[ 0 ].num = 0;

    // Initialize control variables

    Psize = Lsize = 0;
}

int
prog_search(int linenum)
{
    int i = 1, find = 0;

    while (Lines[ i ].num != PROG_BLANKLINE) {
        if (Lines[ i ].num == linenum) {
            find = 1;
            break;
        } else if (Lines[ i ].num > linenum)
            break;
        i++;
    }

    // NOTE: Returns INDEX number of Lines[]
    //        postive number -> found (linenum == Lines[i].num)
    //       negative number -> not found (linenum < Lines[-i].num)

    return find ? i : -i;
}

static void
spacing(FILE *fp)
{
    if (bcode_nextiseol())
        return;
    else if (bcode_nextisdlm()) {
        if (bcode_next() != BCODE_AT)
            return;
    }
    fprintf(fp, " ");
}

int
prog_list(FILE *fp, int color, int startnum, int endnum)
{
    int sidx, eidx, i, loop;
    bcode_t b;
    byte_t *ptr, ch, esc;

    if (Lsize == 0)
        return PROG_SUCCESS;

    sidx = prog_search(startnum);
    if (sidx < 0)
        return PROG_LINENOTFOUND;
    eidx = prog_search(endnum);
    if (eidx < 0)
        return PROG_LINENOTFOUND;

    bcode_start(Program);
    for (i = sidx; i <= eidx; i++) {
        fprintf(fp, "%04d ", Lines[ i ].num);
        bcode_setpc(Lines[ i ].add);
        loop = 1;
        while (loop) {
            bcode_read(&b);
            switch (b.type) {
                case BCODE_TYPE_EOL:
                    loop = 0;
                    break;

                case BCODE_TYPE_NUMBER10:
                    fprintf(fp, "%d", b.num);
                    spacing(fp);
                    break;
```

```
133:
134:                 case BCODE_TYPE_NUMBER16:
135:                     if (b.num & 0xFF00)
136:                         fprintf(fp, "0x%04hX", b.num);
137:                     else
138:                         fprintf(fp, "0x%02hX", b.num);
139:                     spacing(fp);
140:                     break;
141:
142:                 case BCODE_TYPE_STRING:
143:                     fprintf(fp, "\"");
144:                     ptr = b.str;
145:                     while ((ch = *ptr++)) {
146:                         esc = 0;
147:                         if (ch < ' ' || ch == '\\') {
148:                             switch (ch) {
149:                                 case '\007':
150:                                     esc = 'a';
151:                                     break;
152:
153:                                 case '\010':
154:                                     esc = 'b';
155:                                     break;
156:
157:                                 case '\011':
158:                                      esc = 't';
159:                                     break;
160:
161:                                 case '\012':
162:                                     esc = 'n';
163:                                     break;
164:
165:                                 case '\015':
166:                                     esc = 'r';
167:                                     break;
168:
169:                                 case '\033':
170:                                     esc = 'e';
171:                                     break;
172:
173:                                 case '\\':
174:                                     esc = '\\';
175:                                     break;
176:
177:                                 default:
178:                                     esc = 'x';
179:                                     break;
180:                             }
181:                         }
182:                         if (esc) {
183:                             if (esc == 'x')
184:                                 fprintf(fp, "\\x%02X", ch);
185:                             else
186:                                 fprintf(fp, "\\%c", esc);
187:                         } else
188:                             fprintf(fp, "%c", ch);
189:                     }
190:                     fprintf(fp, "\"");
191:                     break;
192:
193:                 case BCODE_TYPE_VARIABLE:
194:                     if (color)
195:                         fprintf(fp, ESCAPE_LBLUE);
196:                     fprintf(fp, "%c", b.idx + 'A');
197:                     if (color)
198:                         fprintf(fp, ESCAPE_DEFAULT);
199:                     spacing(fp);
200:                     break;
201:
202:                 case BCODE_TYPE_KEYWORD:
203:                     if (color)
204:                         fprintf(fp, ESCAPE_LGREEN);
205:                     fprintf(fp, "%s", token_keyword(b.idx));
206:                     if (color)
207:                         fprintf(fp, ESCAPE_DEFAULT);
208:                     spacing(fp);
209:                     break;
210:
211:                 case BCODE_TYPE_DELIMITER:
212:                     if (color && b.inst == BCODE_AT)
213:                         fprintf(fp, ESCAPE_LBLUE);
214:                     fprintf(fp, "%s", token_delimiter(b.idx));
215:                     if (color && b.inst == BCODE_AT)
216:                         fprintf(fp, ESCAPE_DEFAULT);
217:                     if (b.inst == BCODE_RPAREN || b.inst == BCODE_RBRACK)
218:                         if (! bcode_nextisdlm() && ! bcode_nextiseol())
219:                             fprintf(fp, " ");
220:                     break;
221:             }
222:         }
223:         fprintf(fp, "\n");
224:     }
225:     return PROG_SUCCESS;
226: }
227:
228: int
229: prog_delete(int startnum, int endnum)
230: {
231:     int sidx, eidx, dlines, dbytes, mbytes, i;
232:     byte_t *src, *dst;
233:
234:     if (Lsize == 0)
235:         return PROG_LINENOTFOUND;
236:
237:     sidx = prog_search(startnum);
238:     if (sidx < 0)
239:         return PROG_LINENOTFOUND;
240:     eidx = prog_search(endnum);
241:     if (eidx < 0)
242:         return PROG_LINENOTFOUND;
243:
244:     dlines = eidx - sidx + 1;
245:     dbytes = 0;
246:     for (i = sidx; i <= eidx; i++)
247:         dbytes += Lines[ i ].len;
248:
249:     if (eidx != Lsize) {
250:         dst = &Program[ Lines[ sidx ].add ];
251:         src = &Program[ Lines[ eidx + 1 ].add ];
252:         mbytes = 0;
253:         for (i = eidx + 1; i <= Lsize; i++)
254:             mbytes += Lines[ i ].len;
255:
256:         // Transfer tail program
257:         while (mbytes--)
258:             *dst++ = *src++;
259:
260:         // Transfer tail line holder
261:         for  (i = eidx + 1; i <= Lsize; i++) {
262:             Lines[ i - dlines ].num = Lines[ i ].num;
263:             Lines[ i - dlines ].add = Lines[ i ].add - dbytes;
264:             Lines[ i - dlines ].len = Lines[ i ].len;
```

```
265:         }
266:     }
267:
268:     for (i = Lsize - dlines + 1; i <= Lsize; i++) {
269:         Lines[ i ].num = PROG_BLANKLINE;
270:         Lines[ i ].add = 0;
271:         Lines[ i ].len = 0;
272:     }
273:
274:     Psize -= dbytes;
275:     Lsize -= dlines;
276:
277:     return PROG_SUCCESS;
278: }
279:
280: int
281: prog_insert(int linenum, byte_t *src, int len)
282: {
283:     byte_t *dst, *tsrc, *tdst;
284:     int idx, i, start, move;
285:
286:     if (Lsize == PROG_MAXLINE)
287:         return PROG_TOOMANYLINES;
288:     if ((Psize  + len) > PROG_MAXSIZE)
289:         return PROG_SIZEOVER;
290:
291:     idx = prog_search(linenum);
292:     if (idx > 0) {
293:         prog_delete(linenum, linenum);
294:         idx = prog_search(linenum);
295:     }
296:     idx = -idx;
297:
298:     start = 0;
299:     for (i = 1; i < idx; i++) {
300:         start += Lines[ i ].len;
301:     }
302:     move = Psize - start;       // bytes to shift: Program[start .. Psize-1]
303:     dst = &Program[ start ];
304:
305:     if (idx != (Lsize + 1)) {
306:         tsrc = &Program[ Psize - 1 ];
307:         tdst = tsrc + len;
308:         while (move--) {
309:             *tdst-- = *tsrc--;
310:         }
311:         for  (i = Lsize; i >= idx; i--) {
312:             Lines[ i + 1 ].num = Lines[ i ].num;
313:             Lines[ i + 1 ].add = Lines[ i ].add + len;
314:             Lines[ i + 1 ].len = Lines[ i ].len;
315:         }
316:     }
317:
318:     move = len;
319:     while (move--) {
320:         *dst++ = *src++;
321:     }
322:
323:     Lines[ idx ].num = linenum;
324:     Lines[ idx ].add = start;
325:     Lines[ idx ].len = len;
326:     Psize += len;
327:     Lsize++;
328:
329:     return PROG_SUCCESS;
330: }
331:
332: void
333: prog_cmdline(byte_t *src, int size)
334: {
335:     int i;
336:
337:     for (i = 0; i < size; i++)
338:         Program[ PROG_CMDLINE + i ] = *src++;
339:     Lines[ 0 ].num = 0;
340:     Lines[ 0 ].add = PROG_CMDLINE;
341:     Lines[ 0 ].len = size;
342: }
343:
344: int
345: prog_renum(int start, int step)
346: {
347:     int i, add, scan, idx;
348:     bcode_t b;
349:     byte_t *ptr;
350:
351:     Rsize = 0;
352:     for (i = 1; i <= Lsize; i++) {
353:         Pnum = Lines[ i ].num;
354:         add = Lines[ i ].add;
355:         bcode_start(&Program[ add ]);
356:         scan = 1;
357:         while (scan) {
358:             bcode_read(&b);
359:             switch (b.type) {
360:                 case BCODE_TYPE_EOL:
361:                     scan = 0;
362:                     break;
363:
364:                 case BCODE_TYPE_NUMBER10:
365:                 case BCODE_TYPE_NUMBER16:
366:                 case BCODE_TYPE_STRING:
367:                 case BCODE_TYPE_VARIABLE:
368:                 case BCODE_TYPE_DELIMITER:
369:                     break;
370:
371:                 case BCODE_TYPE_KEYWORD:
372:                     if ((b.inst == BCODE_GOSUB || b.inst == BCODE_GOTO) \
373:                             && bcode_nextisnum()) {     // skip GOTO X (computed)
374:                         bcode_read(&b);
375:                         idx = prog_search(b.num);
376:                         if (idx < 0)
377:                             return PROG_LINENOTFOUND;
378:                         if ((Rsize + 1) > PROG_MAXRELOC)
379:                             return PROG_TOOMANYRELOCS;
380:                         Relocs[ Rsize ].idx = idx;
381:                         Relocs[ Rsize ].add = add + b.pos;
382:                         Rsize++;
383:                     }
384:                     break;
385:             }
386:         }
387:     }
388:
389:     // Do renumber
390:
391:     for (i = 1; i <= Lsize; i++)
392:         Lines[ i ].num = start + step * (i - 1);
393:
394:     // Do relocation
395:
396:     for (i = 0; i < Rsize; i++) {
```

```
  397:         ptr = &Program[ Relocs[ i ].add ];
  398:         ptr++;
  399:         ptr[ 0 ] = Lines[ Relocs[ i ].idx ].num & 0xFF;         // little endian,
  400:         ptr[ 1 ] = (Lines[ Relocs[ i ].idx ].num >> 8) & 0xFF;  // no alignment assu
mption
  401:     }
  402:
  403:     return PROG_SUCCESS;
  404: }
  405:
  406: char *
  407: prog_result(int idx)
  408: {
  409:     return Results[ idx ];
  410: }
  411:
  412: #define BYTES_PERLINE    20
  413: void
  414: prog_dumpbytes(void)
  415: {
  416:     int i;
  417:
  418:     printf("%sProgram contents%s\n", ESCAPE_LGREEN, ESCAPE_DEFAULT);
  419:     printf("    Program size = %d bytes\n", Psize);
  420:     for (i = 0; i < Psize; i++) {
  421:         if ((i % BYTES_PERLINE) == 0)
  422:             printf("    0d%05d : ", i);
  423:         printf("%02hX ", Program[ i ]);
  424:         if ((i % BYTES_PERLINE) == (BYTES_PERLINE - 1))
  425:             printf("\n");
  426:     }
  427:     if ((i % BYTES_PERLINE) != (BYTES_PERLINE - 1))
  428:         printf("\n");
  429: }
  430: void
  431: prog_dumplines(void)
  432: {
  433:     int i;
  434:
  435:     printf("%sLine information%s\n", ESCAPE_LGREEN, ESCAPE_DEFAULT);
  436:     printf("    Total %d lines\n", Lsize);
  437:     for (i = 1; i <= Lsize; i++) {
  438:         printf("    Lines[%d].num = %d : Lines[%d].add = %d : ", \
  439:                 i, Lines[ i ].num, i, Lines[ i ].add);
  440:         printf("Lines[%d].len = %d\n", i, Lines[ i ].len);
  441:     }
  442: }
```

```
#include <stdio.h>      // FILE{}, fgets()
#include <unistd.h>     // STD*_FILENO
#include "token.h"
#include "bcode.h"
#include "program.h"

FILE *Debug;

int
main()
{
    char buffer[ 256 ];
    byte_t bytes[ 256 ];
    int bsize, ret, num;
    bcode_t b;

    Debug = stderr;
    token_init();
    prog_init();

    while (fgets(buffer, 256, stdin) != NULL) {
        if (*buffer == 0 || *buffer == '\n')
            break;
        fprintf(Debug, "===== %s", buffer);
        token_source(buffer);
        ret = bsize = bcode_compile(bytes);
        if (bsize < 0) {
            fprintf(Debug, "***** Error: %s\n", bcode_result(ret));
            break;
        }
        fprintf(Debug, ">>>>> Compiled bytecodes [ %d bytes ]\n", bsize);
#ifdef DEBUG
            bcode_dump(bytes);
#endif // DEBUG

        bcode_start(bytes);
        bcode_read(&b);
        if (b.inst == BCODE_LIST) {
            fprintf(Debug, "----------------------------------------\n");
            ret = prog_list(stdout, 1, Lines[1].num, Lines[Lsize].num);
            fprintf(Debug, "----------------------------------------\n");
        } else if (b.inst == BCODE_NEW) {
            prog_init();
            continue;
        } else if (b.type == BCODE_TYPE_NUMBER10) {
            num = b.num;
            if (bcode_next() == BCODE_EOL) {
                ret = prog_delete(num,num);
                if (ret) {
                    fprintf(Debug, "***** Error: %s\n", prog_result(ret));
                    continue;
                }
                fprintf(Debug, ">>>>> Line %d deleted\n", num);
            } else {
                bcode_read(&b);
                ret = prog_insert(num, &bytes[ b.pos ], bsize - 3);
                if (ret) {
                    fprintf(Debug, "***** Error: %s\n", prog_result(ret));
                    continue;
                }
                fprintf(Debug, ">>>>> Line %d inserted\n", num);
            }
        } else {
            fprintf(Debug, "***** Error: illegal program line\n");
            continue;
        }
        prog_dumpbytes();
        printf("\n");
        prog_dumplines();
        ret = 0;
    }
    return 0;
}
```

```
//
//  c o n t a i n e r . h
//
//  BAREMETALHACK.COM --> public domain
//

#ifndef _CONTAINER_H_
#define _CONTAINER_H_

#include "byteword.h"

//
// Container constants
//

#define CONT_MAXARRAY   512
#define CONT_DEFARRAY   10
#define CONT_MAXSTRING  512

//
// Type definition
//

typedef struct {
    int row;
    int col;
} dim_t;

//
// Global variables
//

extern word_t Var[ 26 ];
extern word_t Array[ 26 ][ CONT_MAXARRAY ];
 extern dim_t Asize[ 26 ];
extern byte_t String[ CONT_MAXSTRING ];

//
// Function prototype
//

void cont_init(void);
void cont_dumpvar(void);
void cont_dumparr(void);
void cont_dumpstr(void);

#endif // _CONTAINER_H_
```

```
//
//  c o n t a i n e r . c
//
//  BAREMETALHACK.COM --> public domain
//

#include <stdio.h>          // printf()
#include "container.h"
#include "escape.h"

//
// Global variables
//

word_t Var[ 26 ];
word_t Array[ 26 ][ CONT_MAXARRAY ];
 dim_t Asize[ 26 ];
byte_t String[ CONT_MAXSTRING ];

//
// Container functions
//

void
cont_init(void)
{
    int i, j;

    for (i = 0; i < 26; i++)
        Var[ i ] = 0;

    for (i = 0; i < 26; i++)
        for (j = 0; j < CONT_MAXARRAY; j++)
            Array[ i ][ j ] = 0;

    for (i = 0; i < 26; i++) {
        Asize[ i ].row = 1;
        Asize[ i ].col = CONT_DEFARRAY;
    }

    for (i = 0; i < CONT_MAXSTRING; i++)
        String[ i ] = 0;
}

void
cont_dumpvar(void)
{
    int i;

    printf("%sVariables%s\n", ESCAPE_LGREEN, ESCAPE_DEFAULT);
    for (i = 0; i < 26; i++)
        printf("%c = %6d (0x%04hX)\n", 'A' + i, Var[ i ], Var[ i ]);
}

void
cont_dumparr(void)
{
    int i, j, k;

    printf("%sArrays%s\n", ESCAPE_LGREEN, ESCAPE_DEFAULT);
    for (i = 0; i < 26; i++) {
        printf("DIM %c[%d,%d] = { ", 'A' + i, Asize[ i ].row, Asize[ i ].col);
        for (j = 0; j < Asize[ i ].row; j++) {
            if (Asize[ i ].row > 1)
                printf("{ ");
            for (k = 0; k < Asize[ i ].col; k++) {
                printf("%d", Array[ i ][ Asize[ i ].col * j + k ]);
                if (k != (Asize[ i ].col - 1))
                    printf(",");
            }
            if (Asize[ i ].row > 1)
                printf(" }");
            if (j != (Asize[ i ].row - 1))
                printf(", ");
        }
        printf(" }\n");
    }
}

#define CHARS_PERLINE    20
void
cont_dumpstr(void)
{
    int i = 0, len = 0, cnt, idx;
    char ch;

    printf("%sString @%s\n", ESCAPE_LGREEN, ESCAPE_DEFAULT);
    while(String[ i++ ])
        len++;
    len++;

    idx = 0;
    cnt = len;
    while (len) {
        cnt = (len > CHARS_PERLINE) ? CHARS_PERLINE : len;
        for (i = 0; i < cnt; i++)
            printf("%02hhX ", String[ idx + i ]);
        printf("| ");
        for (i = 0; i < cnt; i++) {
            ch = String[ idx + i ];
            printf("%c", (ch < ' ') ? '?' : ch);
        }
        printf("\n");
        idx += cnt;
        len -= cnt;
    }
}
```

```
 1: //
 2: // f u n c t i o n . h
 3: //
 4: // BAREMETALHACK.COM --> public domain
 5: //
 6:
 7: #ifndef _FUNCTION_H_
 8: #define _FUNCTION_H_
 9:
10: //
11: // Functions
12: //
13:
14:  int func_rnd(void);
15: #ifdef PI
16:  int func_in(int);
17: #endif // PI
18:
19: #endif // _FUNCTION_H_
```

```
//
// f u n c t i o n . c
//
// BAREMETALHACK.COM --> public domain
//

#include <sys/time.h>           // timeval{}, gettimeofday()
#include <stdlib.h>             // srandom(), random()

#ifdef PI
#include "pi_gpio.h"
#endif // PI

//
// BASIC functions
//

static int Seed = 0;

int
func_rnd(void)
{
/*** Classical version using modullo
    static word_t x = 1025;

    x = (257 * x + 1) % 0x8000;
    return (word_t) x;
***/
    struct timeval t;

    if (Seed == 0) {
        gettimeofday(&t, NULL);
        Seed = t.tv_sec;
        srandom(Seed);;
    }
    return random() & 0x7FFF;
}

#ifdef PI
int
func_in(int bit)
{
    return gpio_in(bit);
}
#endif // PI
```

```
//
//  e v a l . h
//
//  BAREMETALHACK.COM --> public domain
//

#ifndef _EVAL_H_
#define _EVAL_H_

#include <setjmp.h>         // jmp_buf{}
#include "container.h"      // dim_t{}

//
// Result codes
//

#define EVAL_SUCCESS        0
#define EVAL_ILLARRAY       1
#define EVAL_ILLFACTOR      2
#define EVAL_ILLINDEX       3
#define EVAL_INDEXOVER      4
#define EVAL_NOASSIGN       5
#define EVAL_NOCOMMA        6
#define EVAL_NOLBRACK       7
#define EVAL_NORBRACK       8
#define EVAL_NOLPAREN       9
#define EVAL_NORPAREN       10
#define EVAL_NOSTRING       11
#define EVAL_DIVBYZERO      12

//
// Function prototype
//

  void eval_init(jmp_buf *);
  void eval_dim(dim_t *);
  void eval_assign(int);
word_t eval(void);
 char *eval_result(int);

#endif // _EVAL_H_
```

```
//
//  e v a l . c
//
//  BAREMETALHACK.COM --> public domain
//

#include <setjmp.h>         // jmp_buf{}, longjmp()
#include "bcode.h"
#include "container.h"
#include "function.h"
#include "eval.h"
#include "debug.h"

//
// File scope variables
//

static jmp_buf *Exit;
static bcode_t Bcode;

static char * Results[] = {
    "OK",                   // EVAL_SUCCESS
    "illegal array",        // EVAL_ILLARRAY
    "illegal factor",       // EVAL_ILLFACTOR
    "illegal index",        // EVAL_ILLINDEX
    "index OVER",           // EVAL_INDEXOVER
    "no assignment",        // EVAL_NOASSIGN
    "no comma",             // EVAL_NOCOMMA
    "no left bracket",      // EVAL_NOLBRAK
    "no right bracket",     // EVAL_NOLBRAK
    "no left parenthesis",  // EVAL_NOLPAREN
    "no right parenthesis", // EVAL_NORPAREN
    "no string",            // EVAL_NOSTRING
    "divided by ZERO"       // EVAL_DIVBYZERO
};

//
// Error handler
//

void
eval_error(int code)
{
    longjmp(*Exit, code);
}

//
// Initializer
//

void
eval_init(jmp_buf *env)
{
    Exit = env;
}

//
// Dimension evaluator
//

void
eval_dim(dim_t *dim)
{
    word_t row = 0, col = 0;

    if (bcode_next() != BCODE_LBRACK)
        eval_error(EVAL_NOLBRACK);
    bcode_skip();
    row = eval();
    if (bcode_next() == BCODE_RBRACK) {
        bcode_skip();
        if (row < 0 || row > CONT_MAXARRAY)
            eval_error(EVAL_ILLINDEX);
        col = row;
        row = 0;
    } else if (bcode_next() == BCODE_COMMA) {
        bcode_skip();
        col = eval();
        if (row < 0 || row > CONT_MAXARRAY)    // row was not checked in 2-D form
            eval_error(EVAL_ILLINDEX);
        if (col < 0 || col > CONT_MAXARRAY)
            eval_error(EVAL_ILLINDEX);
        if (bcode_next() != BCODE_RBRACK)
            eval_error(EVAL_NORBRACK);
        bcode_skip();
    } else
        eval_error(EVAL_ILLARRAY);
    dim->row = row;
    dim->col = col;
}

// Assign value to variable or array
//     var == 0 -> assign to string holder @[]
//     var != 0 -> assign to variable/array (index = var - 1)

void
eval_assign(int var)
{
    bcode_t b;
    byte_t *src, *dst;
    word_t val, idx;
    int array, row, col;
    dim_t dim;

    // String array (@[])

    if (var == 0) {
        if (bcode_next() == BCODE_ASSIGN) {
            bcode_skip();
            if (! bcode_nextisstr())
                eval_error(EVAL_NOSTRING);
            bcode_read(&b);
            src = b.str;
            dst = String;
            while (*src)
                *dst++ = *src++;
            *dst = 0;
            return;
        } else if (bcode_next() == BCODE_LBRACK) {
            bcode_skip();
            idx = eval();
            if (idx < 0 || idx >= CONT_MAXSTRING)
                eval_error(EVAL_ILLINDEX);
            if (bcode_next() != BCODE_RBRACK)
                eval_error(EVAL_NORBRACK);
            bcode_skip();

            if (bcode_next() != BCODE_ASSIGN)
                eval_error(EVAL_NOASSIGN);
            bcode_skip();

            val = eval();
```

```
133:             String[ idx ] = (byte_t) (val & 0xFF);
134:             DMSG("@[%d] = %d", idx, String[idx]);
135:             return;
136:         } else
137:             eval_error(EVAL_NOLBRACK);
138:     }
139:
140:     // Variable or standard array
141:
142:     var--;  // Recover right index
143:     if (bcode_next() == BCODE_LBRACK) {
144:         eval_dim(&dim);
145:         row = dim.row;
146:         col = dim.col;
147:         if (dim.row >= Asize[ var ].row || dim.col >= Asize[ var ].col)
148:             eval_error(EVAL_ILLINDEX);
149:         array = 1;
150:         DMSG("rowsiz = %d : colsiz = %d : row = %d : col = %d", \
151:                 Asize[ var ].row, Asize[ var ].col, row, col);
152:     } else
153:         array = 0;
154:
155:     if (bcode_next() != BCODE_ASSIGN)
156:         eval_error(EVAL_NOASSIGN);
157:     bcode_skip();
158:
159:     // Variable assignment
160:
161:     if (! array) {
162:         val = eval();
163:         Var[ var ] = val;
164:         DMSG("variable %c = %d", var+'A', val);
165:         return;
166:     }
167:
168:     // Array assignment
169:
170:     while (1) {
171:         val = eval();
172:         idx = Asize[ var ].col * row + col;
173:         Array[ var ][ idx ] = val;
174:         DMSG("array %c[%d,%d] = %c[%d] = %d", var+'A', row, col, var+'A', idx, val);
175:         if (bcode_next() == BCODE_COMMA) {
176:             bcode_skip();
177:             col++;
178:             if (col >= Asize[ var ].col) {
179:                 col = 0;
180:                 row++;
181:                 if (row >= Asize[ var ].row)
182:                     eval_error(EVAL_INDEXOVER);
183:             }
184:         } else
185:             break;
186:     }
187:     return;
188: }
189:
190: //
191: // Recursive descent evaluator
192: //
193:
194: static word_t
195: level_zero(void)
196: {
197:     word_t res = 0, v, idx;
198:     dim_t dim;
199:
200:     if (Bcode.type == BCODE_TYPE_NUMBER10) {
201:         res = Bcode.num;
202:         DMSG("number %d", Bcode.num);
203:     } else if (Bcode.type == BCODE_TYPE_NUMBER16) {
204:         res = Bcode.num;
205:         DMSG("number 0x%04hX", Bcode.num);
206:     } else if (Bcode.inst == BCODE_LPAREN) {
207:         res = eval();
208:         if (bcode_next() != BCODE_RPAREN)
209:             eval_error(EVAL_NORPAREN);
210:         bcode_skip();
211:         DMSG("( %d )", res);
212:     } else if (Bcode.type == BCODE_TYPE_VARIABLE) {
213:         v = Bcode.idx;
214:         if (bcode_next() == BCODE_LBRACK) {
215:             eval_dim(&dim);
216:             if (dim.row >= Asize[ v ].row || dim.col >= Asize[ v ].col)
217:                 eval_error(EVAL_ILLINDEX);
218:             idx = Asize[ v ].col * dim.row + dim.col;
219:             res = Array[ v ][ idx ];
220:             DMSG("array %c[%d,%d] = %c[%d] = %d",
221:                  v+'A', dim.row, dim.col, v+'A', idx, res);
222:         } else {
223:             res = Var[ v ];
224:             DMSG("variable %c = %d", v+'A', res);
225:         }
226:     } else if (Bcode.inst == BCODE_AT) {
227:         if (bcode_next() != BCODE_LBRACK)
228:             eval_error(EVAL_NOLBRACK);
229:         bcode_skip();
230:         idx = eval();
231:         if (bcode_next() != BCODE_RBRACK)
232:             eval_error(EVAL_NORBRACK);
233:         bcode_skip();
234:         if (idx < 0 || idx >= CONT_MAXSTRING)
235:             eval_error(EVAL_ILLINDEX);
236:         res = (word_t) String[ idx ];
237:         DMSG("@[ %d ] = %d", idx, res);
238:     } else if (Bcode.inst == BCODE_RND) {
239:         if (bcode_next() != BCODE_LPAREN)
240:             eval_error(EVAL_NOLPAREN);
241:         bcode_skip();
242:         if (bcode_next() != BCODE_RPAREN)
243:             eval_error(EVAL_NORPAREN);
244:         bcode_skip();
245:         res = func_rnd();
246:         DMSG("rnd() = %d", res);
247: #ifdef PI
248:     } else if (Bcode.inst == BCODE_IN) {
249:         if (bcode_next() != BCODE_LPAREN)
250:             eval_error(EVAL_NOLPAREN);
251:         bcode_skip();
252:         idx = eval();
253:         if (bcode_next() != BCODE_RPAREN)
254:             eval_error(EVAL_NORPAREN);
255:         bcode_skip();
256:         res = func_in((int) idx);
257:         DMSG("in(%d) = %d", idx, res);
258: #endif // PI
259:     } else
260:         eval_error(EVAL_ILLFACTOR);
261:     return res;
262: }
263:
264: static word_t
```

```
265: level_one(void)
266: {
267:     word_t unary, res;
268:
269:     if (Bcode.inst == BCODE_SUB) {
270:         bcode_read(&Bcode);
271:         unary = level_zero();
272:         DMSG("-%d", unary);
273:         res = -unary;
274:     } else if (Bcode.inst == BCODE_ADD) {
275:         bcode_read(&Bcode);
276:         unary = level_zero();
277:         DMSG("+%d", unary);
278:         res = unary;
279:     } else
280:         res = level_zero();
281:     return res;
282: }
283:
284: static word_t
285: level_two(void)
286: {
287:     int op;
288:     word_t first, second;
289:
290:     first = level_one();
291:     while (1) {
292:         op = bcode_next();
293:         if (op == BCODE_MUL || op == BCODE_DIV || op == BCODE_MOD) {
294:             bcode_skip();
295:             bcode_read(&Bcode);
296:             second = level_one();
297:             switch (op) {
298:                 case BCODE_MUL:
299:                     DMSG("%d * %d", first, second);
300:                     first = first * second;
301:                     break;
302:
303:                 case BCODE_DIV:
304:                     if (second == 0)
305:                         eval_error(EVAL_DIVBYZERO);
306:                     DMSG("%d / %d", first, second);
307:                     first = first / second;
308:                     break;
309:
310:                 case BCODE_MOD:
311:                     if (second == 0)
312:                         eval_error(EVAL_DIVBYZERO);
313:                     DMSG("%d %% %d", first, second);
314:                     first = first % second;
315:                     break;
316:             }
317:         } else
318:             break;
319:     }
320:     return first;
321: }
322:
323: static word_t
324: level_three(void)
325: {
326:     int op;
327:     word_t first, second;
328:
329:     first = level_two();
330:     while (1) {
331:         op = bcode_next();
332:         if (op == BCODE_ADD || op == BCODE_SUB) {
333:             bcode_skip();
334:             bcode_read(&Bcode);
335:             second = level_two();
336:             switch (op) {
337:                 case BCODE_ADD:
338:                     DMSG("%d + %d", first, second);
339:                     first = first + second;
340:                     break;
341:
342:                 case BCODE_SUB:
343:                     DMSG("%d - %d", first, second);
344:                     first = first - second;
345:                     break;
346:             }
347:         } else
348:             break;
349:     }
350:     return first;
351: }
352:
353: static word_t
354: level_four(void)
355: {
356:     int op;
357:     word_t first, second;
358:
359:     first = level_three();
360:     while (1) {
361:         op = bcode_next();
362:         if (op == BCODE_EQUAL || op == BCODE_GEQUAL ||
363:             op == BCODE_GTHAN || op == BCODE_LEQUAL ||
364:             op == BCODE_LTHAN || op == BCODE_NEQUAL) {
365:             bcode_skip();
366:             bcode_read(&Bcode);
367:             second = level_three();
368:             switch (op) {
369:                 case BCODE_EQUAL:
370:                     DMSG("%d == %d", first, second);
371:                     first = (first == second) ? 1 : 0;
372:                     break;
373:
374:                 case BCODE_GEQUAL:
375:                     DMSG("%d >= %d", first, second);
376:                     first = (first >= second) ? 1 : 0;
377:                     break;
378:
379:                 case BCODE_GTHAN:
380:                     DMSG("%d > %d", first, second);
381:                     first = (first > second) ? 1 : 0;
382:                     break;
383:
384:                 case BCODE_LEQUAL:
385:                     DMSG("%d <= %d", first, second);
386:                     first = (first <= second) ? 1 : 0;
387:                     break;
388:
389:                 case BCODE_LTHAN:
390:                     DMSG("%d < %d", first, second);
391:                     first = (first < second) ? 1 : 0;
392:                     break;
393:
394:                 case BCODE_NEQUAL:
395:                     DMSG("%d != %d", first, second);
396:                     first = (first != second) ? 1 : 0;
```

```
397:                     break;
398:             }
399:         } else
400:             break;
401:     }
402:     return first;
403: }
404:
405: static word_t
406: level_five(void)
407: {
408:     int op;
409:     word_t first, second;
410:
411:     first = level_four();
412:     while (1) {
413:         op = bcode_next();
414:         if (op == BCODE_AND || op == BCODE_OR) {
415:             bcode_skip();
416:             bcode_read(&Bcode);
417:             second = level_four();
418:             switch (op) {
419:                 case BCODE_AND:
420:                     DMSG("%d & %d", first, second);
421:                     first = first & second;
422:                     break;
423:
424:                 case BCODE_OR:
425:                     DMSG("%d | %d", first, second);
426:                     first = first | second;
427:                     break;
428:             }
429:         } else
430:             break;
431:     }
432:     return first;
433: }
434:
435: word_t
436: eval(void)
437: {
438:     word_t res;
439:
440:     bcode_read(&Bcode);
441:     if (Bcode.inst == BCODE_EOL)
442:         return 0;
443:     res = level_five();
444:     DMSG("--> %d (0x%04X)", res, res);
445:     return res;
446: }
447:
448: char *
449: eval_result(int idx)
450: {
451:     return Results[ idx ];
452: }
```

```
    1: #include <stdio.h>      // fprintf()
    2: #include <setjmp.h>     // setjmp()
    3:
    4: #include "token.h"
    5: #include "bcode.h"
    6: #include "eval.h"
    7:
    8: FILE *Debug;
    9: jmp_buf Recovery;
   10:
   11: int
   12: main()
   13: {
   14:     char buffer[ 256 ], *ptr;
   15:     byte_t bytes[ 256 ];
   16:     int ret;
   17:     word_t val;
   18:
   19:     Debug = stderr;
   20:     token_init();
   21:     eval_init(&Recovery);
   22:     while (1) {
   23:         fprintf(stdout, "\n");
   24:         fprintf(Debug, "\n");
   25:         if ((ret = setjmp(Recovery)) != 0) {
   26:             fprintf(stdout, "*** eval() error: %s\n", eval_result(ret));
   27:             continue;
   28:         }
   29:         ptr = fgets(buffer, 256, stdin);
   30:         if (ptr == 0 || *buffer == 0 || *buffer == '\n')
   31:             break;
   32:         token_source(buffer);
   33:         ret = bcode_compile(bytes);
   34:         if (ret < 0) {
   35:             fprintf(stdout, "*** bcode_compile() error: %s ***\n", bcode_result(-ret
));
   36:             continue;
   37:         }
   38:         bcode_start(bytes);
   39:         val = eval();
   40:         fprintf(stdout, "eval() returns %d\n", val);
   41:     }
   42:     return 0;
   43: }
```

```
 1: #include <stdio.h>      // fprintf()
 2: #include <setjmp.h>     // setjmp()
 3:
 4: #include "bcode.h"
 5: #include "eval.h"
 6:
 7: FILE *Debug;
 8: jmp_buf Recovery;
 9:
10: int
11: main()
12: {
13:     char buffer[ 256 ], *ptr;
14:     byte_t bytes[ 256 ];
15:     int ret;
16:     word_t val;
17:     bcode_t b;
18:
19:     Debug = stderr;
20:     eval_init(&Recovery);
21:     while (1) {
22:         printf("\n");
23:         fprintf(Debug, "\n");
24:         if ((ret = setjmp(Recovery)) != 0) {
25:             printf("*** eval() error: %s\n", eval_result(ret));
26:             continue;
27:         }
28:         ptr = fgets(buffer, 256, stdin);
29:         if (ptr == 0 || *buffer == 0 || *buffer == '\n')
30:             break;
31:         token_source(buffer);
32:         ret = bcode_compile(bytes);
33:         if (ret < 0) {
34:             printf("*** bcode_compile() error: %s\n", bcode_result(-ret));
35:             continue;
36:         }
37:         bcode_start(bytes);
38:         bcode_read(&b);
39:         if (b.inst == BCODE_INQ) {
40:             val = eval();
41:             printf("eval() returns %d\n", val);
42:         } else if (b.type == BCODE_TYPE_VARIABLE)
43:             eval_assign(b.idx + 1);
44:         else if (b.inst == BCODE_AT)
45:             eval_assign(0);
46:         else
47:             printf("*** Syntax error\n");
48:     }
49:     return 0;
50: }
```

```
//
//  b a s i c . h
//
//  BAREMETALHACK.COM --> public domain
//

#ifndef _BASIC_H_
#define _BASIC_H_

#include <setjmp.h>             // jmp_buf{}

//
// Program information
//

#ifdef PI
#define BASIC_NAME              "antBASIC for Pi"
#else
#define BASIC_NAME              "antBASIC for Unix"
#endif // PI

// Version number

#define BASIC_MAJOR             1
#define BASIC_MINOR             0
#define BASIC_PATCH             4

//
// BASIC constants
//

#define BASIC_MAXLINECHAR       512     // Maximum characters on command line
#define BASIC_MAXSTACKDEPTH     8       // Maximum depth of GOSUB/FOR stack
#define BASIC_LINESTART         100     // RENUM default start line number
#define BASIC_LINESTEP          10      // RENUM default line step

#define BASIC_MODERUN           0       // Normal execution mode (RUN)
#define BASIC_MODEDIRECT        1       // Direct execution from command line
#define BASIC_MODELOAD          2       // Program load mode (LOAD/MERGE)
#define BASIC_MODESHELL         4       // Execution from the shell

//
// Return codes
//

#define BASIC_SUCCESS           0
#define BASIC_DIRECTDENY        1
#define BASIC_DIRNOTFOUND       2
#define BASIC_FILEIOERROR       3
#define BASIC_ILLARRAY          4
#define BASIC_ILLRANGE          5
#define BASIC_NOARRAY           6
#define BASIC_NOASSIGN          7
#define BASIC_NOCONTROLVAR      8
#define BASIC_NODIRNAME         9
#define BASIC_NOLINENUM         10
#define BASIC_NOFILENAME        11
#define BASIC_NOSTACK           12
#define BASIC_NOVARIABLE        13
#define BASIC_STACKOVER         14
#define BASIC_SYNTAXERROR       15

// Error code group BIT

#define BASIC_BCODE_ERROR       0b00100000
#define BASIC_PROG_ERROR        0b01000000
#define BASIC_GPIO_ERROR        0b10000000

//
// Type definitions
//

typedef struct {
    int lidx;                   // Line index number
    int radd;                   // Return address
} gstack_t;

typedef struct {
    int lidx;                   // Line index number
    int ladd;                   // Looping entry address
    int vidx;                   // Variable index number
    int ecnt;                   // End count
} fstack_t;

//
// Global variable
//

extern int Lnum;                // Line number currently executing
extern int Mode;                // Current mode (NORMAL/DIRECT/LOAD/SHELL)
extern int Status;              // Exit status (in the case of SHELL mode)
extern char Text[];             // Source text line buffer

//
// Function prototype
//

  void basic_init(jmp_buf *);
  void basic_error(int);
   int basic_readline(char *, char *, int);
   int basic_command(int);
   int basic_load(char *);
   int basic_exec(void);
 char *basic_result(int);

#endif  // _BASIC_H_
```

```
//
//  b a s i c . c
//
//  BAREMETALHACK.COM --> public domain
//

#include <stdio.h>              // FILE{}, stdin, fgets(), fflush()
#include <setjmp.h>             // jmp_buf{}, longjmp()
#include <string.h>             // strlen(), strncpy()
#include <stdlib.h>             // atoi(), free()
#include <dirent.h>             // DIR{}, dirent{}, scandir()
#include <unistd.h>             // usleep()

#ifdef READLINE
#include <readline/readline.h>  // readline(), rl_*()
#include <readline/history.h>   // add_history()
#endif // READLINE

#include "basic.h"
#include "token.h"
#include "bcode.h"
#include "eval.h"
#include "program.h"
#include "container.h"
#include "escape.h"
#include "sigint.h"
#include "debug.h"

#ifdef PI
#include "pi_gpio.h"
#endif // PI

//
// Global variable
//

 int Lnum;                  // Current executing line number
 int Mode;                  // Current executing mode (NORMAL/DIRECT/LOAD/SHELL)
 int Status = 0;            // Exit status which will be returned to the shell
char Text[ TOKEN_MAXBUF ];  // Source text line buffer

//
// File scope variables
//

static jmp_buf * Exit;
      static int Lidx;
      static int Gsize, Fsize;
 static gstack_t Gstack[ BASIC_MAXSTACKDEPTH ];
 static fstack_t Fstack[ BASIC_MAXSTACKDEPTH ];

static char * Results[] = {
    "BASIC \"ok\"",                     // BASIC_SUCCESS
    "BASIC \"direct exec is denied\"",  // BASIC_DIRECTDENY
    "BASIC \"directory not found\"",    // BASIC_DIRNOTFOUND
    "BASIC \"file I/O error\"",         // BASIC_FILEIOERROR
    "BASIC \"illegal array\"",          // BASIC_ILLARRAY
    "BASIC \"illegal range\"",          // BASIC_ILLRANGE
    "BASIC \"no array\"",               // BASIC_NOARRAY
    "BASIC \"no assignment\"",          // BASIC_NOASSIGN
    "BASIC \"no control variable\"",    // BASIC_NOCONTROLVAR
    "BASIC \"no directroy name\"",      // BASIC_NODIRNAME
    "BASIC \"no line number\"",         // BASIC_NOLINENUM
    "BASIC \"no filename\"",            // BASIC_NOFILENAME
    "BASIC \"no stack\"",               // BASIC_NOSTACK
    "BASIC \"no variable\"",            // BASIC_NOVARIABLE
    "BASIC \"stack is overflowed\"",    // BASIC_STACKOVER
    "BASIC \"syntax error\""            // BASIC_SYNTAXERROR
};

static char *help_container[] = {
    "CONTAINERS",       "",
    "Numbers",          "Signed 16bit integer (range from -32768 to 32767)$" \
                        "Decimal or hexadecimal (0x prefix is needed)$" \
                        "    ex. 1234, -1234, 0xABCD, 0xEF",
    "Strings",          "8bit ASCII characters surrounded by double quotation$" \
                        "Escaped special characters are as follows:$" \
                        "    \\a BEL, \\b BS, \\t TAB, \\n LF, \\r CR, \\e ESC, \\\\ b
ackslash$" \
                        "Special array @ holds string$" \
                        "    ex. @=\"hello!\":@[0]=@[0]-0x20:print @ -> Hello!$" \
                        "        @[0]=33:@[1]=7:@[2]=0:print @ -> ! with alarm",
    "Variables",        "Vaiable A to Z holds integer: ex. A=123:B=A+0x1234",
    "Arrays",           "Array A[] to Z[] holds integer (index starts from ZERO)$" \
                        "Two-dimensional array form is X[column,row]$" \
                        "    ex. DIM A[1],B[2,3]:A[0]=1:B[0,0]=0,1,2,3,4,5$" \
                        "        A[0] -> 1, B[0,2] -> 2, B[1,0] -> 3, B[1,2] -> 5",
    0
};

static char *help_operator[] = {
    "OPERATORS",        "Precedence (high to low)",
    "Unary",            "-xxx, +xxx",
    "Mul/Div/Mod",      "*, /, %",
    "Add/Sub",          "+, -",
    "Condition",        "==, !=, <, <=, >, >=",
    "Bitwise",          "&, |",
    0
};

static char *help_statement[] = {
    "STATEMENTES",      "",
    "CLS",              "Clear screen",
    "COLOR",            "Define color attribute$" \
                        "    Color codes: 0)Black, 1)Red, 2)Green, 3)Yellow$" \
                        "        4)Blue, 5)Magenta, 6)Cyan, 7)White, +10)Bright$" \
                        "    Fore-graound, Back-ground: COLOR(F,B)",
    "DIM",              "Define array size: DIM[ col, row ]$" \
                        "    NOTE: array size limitation, col * row <= 512",
    "END",              "Terminate program",
    "FOR/NEXT",         "Iterate statements between FOR and NEXT$" \
                        "    ex. S=0:FOR A=1 TO 10:S=S+A:NEXT$" \
                        "    NOTE: increment step is fixed to ONE",
    "GOTO",             "Jump to specified line number: ex. GOTO 100, GOTO X",
    "GOSUB/RETURN",     "Call subroutine / return to caller: ex. GOSUB 200, GOSUB Y",
    "IF",               "Conditional execution. If the expression immediately after IF
$" \
                        "is not zero, the following statement(s) will be executed.$" \
                        "    ex. IF A>=0x61 @[0]=A-0x20:@[1]=0:PRINT @",
#ifdef PI
    "IN",               "Read GPIO (B 1-14) bit level: IN(B) -> 0|1",
#endif // PI
    "INPUT",            "Input data from user: number) INPUT A, string) INPUT @",
    "LOCATE",           "Locate cursor (left-upper-corner = [0,0]): LOCATE(X,Y)",
#ifdef PI
    "OUT",              "Set GPIO (B 1-14) bit level (L 0 GND|1 Vdd): OUT(B,L)",
    "OUTHZ",            "Set GPIO (B 1-14) bit level (L 0 GND|1 HiZ),$" \
                        "    Internal pull-up (P 0 None|1 Pull): OUTHZ(B,S,P)",
#endif // PI
    "PRINT",            "Print data$" \
                        "    integer: immediate value, variable, array$" \
                        "    string: @$" \
```

```
  131:                       "    separator: semicolon=no spacing, comma=tabulation$" \
  132:                       "      PRINT \"H\";\"I\";\"!\" -> HI!$" \
  133:                       "    hexadecimal format: HEX2(xxx), HEX4(xxxxx)",
  134:     "REM",            "Remark: REM, REM \"This is comment string\"",
  135:     "RND",            "Returns random number (0 to 32767): RND()",
  136:     "MSLEEP",         "Suspend execution for MILLI-seconds: SLEEP(1000) -> 1sec",
  137:     "USLEEP",         "Suspend execution for MICRO-seconds: MLEEP(1000) -> 1usec",
  138:     0
  139: };
  140:
  141: static char *help_command[] = {
  142:     "COMMANDS",       "",
  143:     "CLEAR",          "Clear containers",
  144:     "CLS",            "Clear screen",
  145:     "DELETE",         "Delete statement(s): DELETE 100, DELETE 210,290",
  146:     "DUMP",           "Dump containers: DUMP (all), DUMP V (variables), DUMP A (arra
ys),$" \
  147:                       "     DUMP S (string), DUMP L (lines), DUMP B (bytecodes)",
  148:     "EDIT",           "Edit a program line using GNU Readline input editor: EDIT 120
",
  149:     "END",            "Quit antBASIC",
  150:     "FILES",          "List files in current working directory",
  151:     "FREE",           "Display memory usage",
  152:     "HELP",           "Display help information",
  153:     "LIST",           "List all or part of program: LIST, LIST 100, LIST 210,330",
  154:     "LOAD",           "Load a source file into memory: LOAD \"example/hello.bas\"",
  155:     "MERGE",          "Merge an additional file into memory: MERGE \"mylib/addon.bas
\"",
  156:     "NEW",            "Clear program",
  157:     "RENUM",          "Renumber program: RENUM, RENUM start, RENUM start,step",
  158:     "RUN",            "Start-up program: CONTROL-C aborts the program",
  159:     "SAVE",           "Save program list to a file: SAVE \"myprogram.bas\"",
  160:     0
  161: };
  162:
  163: //
  164: // Initializer
  165: //
  166:
  167: void
  168: basic_init(jmp_buf *env)
  169: {
  170:     Exit = env;
  171:     Lnum = Lidx = Gsize = Fsize = 0;
  172: }
  173:
  174: //
  175: // Helper functions
  176: //
  177:
  178: void
  179: basic_error(int code)
  180: {
  181:     longjmp(*Exit, code);
  182: }
  183:
  184: // Returns: length of the line (0 = empty line), -1 = EOF
  185: // NOTE: a line longer than (size - 1) characters is truncated.
  186:
  187: int
  188: basic_readline(char *msg, char *buf, int size)
  189: {
  190:     char *ptr;
  191:     int len;
  192:
  193: #ifdef READLINE
  194:     ptr = readline(msg);
  195:     if (ptr == NULL) {          // EOF (^D)
  196:         buf[ 0 ] = 0;
  197:         return -1;
  198:     }
  199:     len = snprintf(buf, size, "%s", ptr);   // always NUL terminated
  200:     if (len >= size)
  201:         len = size - 1;
  202:     if (len > 0)
  203:         add_history(ptr);       // do not record empty lines
  204:     free(ptr);
  205: #else
  206:     printf("%s", msg);
  207:     fflush(stdout);
  208:     ptr = fgets(buf, size, stdin);
  209:     if (ptr == NULL) {          // EOF
  210:         buf[ 0 ] = 0;
  211:         return -1;
  212:     }
  213:     while (*ptr) {
  214:         if (*ptr == '\n') {
  215:             *ptr = 0;
  216:             break;
  217:         }
  218:         ptr++;
  219:     }
  220:     len = strlen(buf);
  221: #endif // READLINE
  222:     return len;
  223: }
  224:
  225: //
  226: // BASIC statements
  227: //
  228:
  229: static void
  230: stat_rem(void)
  231: {
  232:     bcode_t b;
  233:
  234:     while (bcode_next() != BCODE_EOL) {
  235:         bcode_read(&b);
  236:     }
  237:     DMSG(">>> %05d REM", Lnum);
  238: }
  239:
  240: static int
  241: stat_end(void)
  242: {
  243:     bcode_t b;
  244:     word_t num;
  245:
  246:     if (bcode_nextiseol())
  247:         num = 0;
  248:     else if (bcode_nextisvar()) {
  249:         bcode_read(&b);
  250:         num = Var[ b.idx ];
  251:     } else
  252:         num = eval();
  253:
  254:     return num;
  255: }
  256:
  257: static void
  258: stat_dim(void)
  259: {
```

```
   260:     bcode_t b;
   261:     int v;
   262:     dim_t size;
   263:
   264:     while (1) {
   265:         bcode_read(&b);
   266:         if (b.type != BCODE_TYPE_VARIABLE)
   267:             basic_error(BASIC_NOARRAY);
   268:         v = b.idx;
   269:
   270:         eval_dim(&size);
   271:         if (size.row < 0 || size.col <= 0)
   272:             basic_error(BASIC_ILLRANGE);
   273:         if (size.row == 0) {
   274:             size.row = 1;
   275:             if (size.col > CONT_MAXARRAY)
   276:                 basic_error(BASIC_ILLRANGE);
   277:         } else if (size.row * size.col > CONT_MAXARRAY)
   278:             basic_error(BASIC_ILLRANGE);
   279:         Asize[ v ].row = size.row;
   280:         Asize[ v ].col = size.col;
   281:         DMSG(">>> %05d DIM %c[%d,%d]\n", Lnum, v+'A', Asize[ v ].row, Asize[ v ].col
);
   282:         if (bcode_next() == BCODE_COLON || bcode_next() == BCODE_EOL)
   283:             return;
   284:         else if (bcode_next() == BCODE_COMMA)
   285:             bcode_skip();
   286:         else
   287:             basic_error(BASIC_ILLARRAY);
   288:     }
   289: }
   290:
   291: static void
   292: stat_print(void)
   293: {
   294:     bcode_t b;
   295:     int pos = 0, len, sp, newline = 1;
   296:     word_t res;
   297:
   298:     while (1) {
   299:         if (bcode_next() == BCODE_COLON || bcode_next() == BCODE_EOL)
   300:             break;
   301:         else if (bcode_next() == BCODE_STRING) {
   302:             bcode_read(&b);
   303:             len = printf("%s", b.str);
   304:             pos += len;
   305:             newline = 1;
   306:         } else if (bcode_next() == BCODE_AT && \
   307:                 bcode_twoahead() != BCODE_LBRACK) {
   308:             bcode_skip();
   309:             len = printf("%s", String);
   310:             pos += len;
   311:             newline = 1;
   312:         } else if (bcode_next() == BCODE_HEX2) {
   313:             bcode_skip();
   314:             if (bcode_next() != BCODE_LPAREN)
   315:                 basic_error(BASIC_SYNTAXERROR);
   316:             bcode_skip();
   317:             res = eval();
   318:             if (bcode_next() != BCODE_RPAREN)
   319:                 basic_error(BASIC_SYNTAXERROR);
   320:             bcode_skip();
   321:             len = printf("%02hhX", (unsigned char) res);
   322:             pos += len;
   323:             newline = 1;
   324:         } else if (bcode_next() == BCODE_HEX4) {
   325:             bcode_skip();
   326:             if (bcode_next() != BCODE_LPAREN)
   327:                 basic_error(BASIC_SYNTAXERROR);
   328:             bcode_skip();
   329:             res = eval();
   330:             if (bcode_next() != BCODE_RPAREN)
   331:                 basic_error(BASIC_SYNTAXERROR);
   332:             bcode_skip();
   333:             len = printf("%04hX", res);
   334:             pos += len;
   335:             newline = 1;
   336:         } else {
   337:             res = eval();
   338:             len = printf("%d", (int) res);
   339:             pos += len;
   340:             newline = 1;
   341:         }
   342:         if (bcode_next() == BCODE_COMMA) {
   343:             bcode_skip();
   344:             sp = 8 - (pos % 8);
   345:             if (sp == 8)
   346:                 sp = 0;
   347:             pos += sp;
   348:             while (sp--)
   349:                 printf(" ");
   350:             newline = 1;
   351:         } else if (bcode_next() == BCODE_SCOLON) {
   352:             bcode_skip();
   353:             newline = 0;
   354:         }
   355:     }
   356:     if (newline)
   357:         printf("\n");
   358:     fflush(stdout);
   359:     DMSG(">>> %05d PRINT <<<", Lnum);
   360: }
   361:
   362: static void
   363: stat_input(void)
   364: {
   365:     int len;
   366:     bcode_t b;
   367:     word_t val;
   368:     char buff[ BASIC_MAXLINECHAR ];
   369:
   370:     if (bcode_next() == BCODE_AT) {
   371:         bcode_skip();
   372:         basic_readline("? ", (char *) String, BASIC_MAXLINECHAR);
   373:         DMSG(">>> %05d INPUT @", Lnum);
   374:     } else if (bcode_nextisvar()) {
   375:         bcode_read(&b);
   376:         len = basic_readline("? ", buff, BASIC_MAXLINECHAR);
   377:         if (len <= 0)               // empty line or EOF
   378:             val = 0;
   379:         else
   380:             val = (word_t) atoi(buff);
   381:         Var[ b.idx ] = val;
   382:         DMSG(">>> %05d INPUT %c = %d\n", Lnum, 'A' + b.idx, val);
   383:     } else
   384:         basic_error(BASIC_NOVARIABLE);
   385: }
   386:
   387: static int
   388: stat_goto(void)
   389: {
   390:     bcode_t b;
```

```
391:     word_t num = 0;
392:
393:     if (bcode_nextisvar()) {
394:         bcode_read(&b);
395:         num = Var[ b.idx ];
396:     } else
397:         num = eval();
398:     DMSG(">>> %05d GOTO %d <<<", Lnum, num);
399:     return num;
400: }
401:
402: static int
403: stat_gosub(void)
404: {
405:     bcode_t b;
406:     word_t num = 0;
407:
408:     if (bcode_nextisvar()) {
409:         bcode_read(&b);
410:         num = Var[ b.idx ];
411:     } else
412:         num = eval();
413:
414:     if (Gsize == BASIC_MAXSTACKDEPTH)
415:         basic_error(BASIC_STACKOVER);
416:     Gstack[ Gsize ].lidx = Lidx;
417:     Gstack[ Gsize ].radd = bcode_getpc();
418:     Gsize++;
419:     DMSG(">>> %05d GOSUB %d", Lnum, num);
420:     return num;
421: }
422:
423: static void
424: stat_return(void)
425: {
426:     if (Gsize == 0)
427:         basic_error(BASIC_NOSTACK);
428:     else
429:         Gsize--;
430:     DMSG(">>> %05d RETURN", Lnum);
431: }
432:
433: static int
434: stat_if(void)
435: {
436:     word_t res;
437:
438:     res = eval();
439:     res = (res != 0) ? 1 : 0;
440:     DMSG(">>> %05d IF %d", Lnum, res);
441:     return res;
442: }
443:
444: static void
445: stat_for(void)
446: {
447:     bcode_t b;
448:     int var;
449:     word_t start, end;
450:
451:     if (! bcode_nextisvar())
452:         basic_error(BASIC_NOCONTROLVAR);
453:     bcode_read(&b);
454:     var = b.idx;
455:
456:     if (bcode_next() != BCODE_ASSIGN)
457:         basic_error(BASIC_NOASSIGN);
458:     bcode_skip();
459:     start = eval();
460:
461:     if (bcode_next() != BCODE_TO)
462:         basic_error(BASIC_SYNTAXERROR);
463:     bcode_skip();
464:     end = eval();
465:
466:     if (start > end)
467:         basic_error(BASIC_ILLRANGE);
468:     if (Fsize == BASIC_MAXSTACKDEPTH)
469:         basic_error(BASIC_STACKOVER);
470:
471:     Var[ var ] = start;
472:     Fstack[ Fsize ].lidx = Lidx;
473:     Fstack[ Fsize ].ladd = bcode_getpc();
474:     Fstack[ Fsize ].vidx = var;
475:     Fstack[ Fsize ].ecnt = (int) end;
476:     Fsize++;
477:     DMSG(">>> %05d FOR lidx=%d : ladd=%d : var '%c' : start=%d : end=%d\n", \
478:             Lnum, Lidx, bcode_getpc(), var + 'A', start, end);
479: }
480:
481: static void
482: stat_next(void)
483: {
484:     if (Fsize == 0)
485:         basic_error(BASIC_NOSTACK);
486:     DMSG(">>> %05d NEXT", Lnum);
487: }
488:
489: static void
490: stat_cls(void)
491: {
492:     printf(ESCAPE_HOME ESCAPE_CLS);
493:     DMSG(">>> %05d CLS", Lnum);
494: }
495:
496: static void
497: stat_color(void)
498: {
499:     word_t fc = -1, bc = -1;
500:
501:     if (bcode_next() != BCODE_LPAREN)
502:         basic_error(BASIC_SYNTAXERROR);
503:     bcode_skip();
504:     fc = eval();
505:     if (bcode_next() == BCODE_COMMA) {
506:         bcode_skip();
507:         bc = eval();
508:     }
509:     if (bcode_next() != BCODE_RPAREN)
510:         basic_error(BASIC_SYNTAXERROR);
511:     bcode_skip();
512:
513:     if (fc >= 0 && fc <= 7) {
514:         printf("\e[%dm", fc + 30);
515:     } else if (fc >= 10 && fc <= 17) {
516:         printf("\e[%dm", fc + 80);
517:     }
518:     if (bc >= 0 && bc <=7) {
519:         printf("\e[%dm", bc + 40);
520:     } else if (bc >= 10 && bc <= 17) {
521:         printf("\e[%dm", bc + 90);
522:     }
```

```
523:     DMSG(">>> %05d COLOR(%d,%d)", Lnum, fc, bc);
524: }
525:
526: static void
527: stat_locate(void)
528: {
529:     word_t x, y;
530:
531:     if (bcode_next() != BCODE_LPAREN)
532:         basic_error(BASIC_SYNTAXERROR);
533:     bcode_skip();
534:     x = eval();
535:     if (bcode_next() != BCODE_COMMA)
536:         basic_error(BASIC_SYNTAXERROR);
537:     bcode_skip();
538:     y = eval();
539:     if (bcode_next() != BCODE_RPAREN)
540:         basic_error(BASIC_SYNTAXERROR);
541:     bcode_skip();
542:
543:     printf("\e[%d;%dH", y, x);
544:     DMSG(">>> %05d LOCATE(%d,%d)", Lnum, x, y);
545: }
546:
547: static void
548: stat_usleep(void)
549: {
550:     word_t val;
551:
552:     if (bcode_next() != BCODE_LPAREN)
553:         basic_error(BASIC_SYNTAXERROR);
554:     bcode_skip();
555:     val = eval();
556:     if (bcode_next() != BCODE_RPAREN)
557:         basic_error(BASIC_SYNTAXERROR);
558:     bcode_skip();
559: #ifdef PI
560:     gpio_usleep(val);
561: #else
562:     usleep(val);
563: #endif // PI
564:     DMSG(">>> %05d USLEEP(%d)", Lnum, val);
565: }
566:
567: static void
568: stat_msleep(void)
569: {
570:     word_t val;
571:
572:     if (bcode_next() != BCODE_LPAREN)
573:         basic_error(BASIC_SYNTAXERROR);
574:     bcode_skip();
575:     val = eval();
576:     if (bcode_next() != BCODE_RPAREN)
577:         basic_error(BASIC_SYNTAXERROR);
578:     bcode_skip();
579:     usleep(val * 1000);
580:     DMSG(">>> %05d MSLEEP(%d)", Lnum, val);
581: }
582:
583: #ifdef PI
584: static void
585: stat_out(void)
586: {
587:     word_t bit, val;
588:     int res;
589:
590:     if (bcode_next() != BCODE_LPAREN)
591:         basic_error(BASIC_SYNTAXERROR);
592:     bcode_skip();
593:     bit = eval();
594:     if (bcode_next() != BCODE_COMMA)
595:         basic_error(BASIC_SYNTAXERROR);
596:     bcode_skip();
597:     val = eval();
598:     if (bcode_next() != BCODE_RPAREN)
599:         basic_error(BASIC_SYNTAXERROR);
600:     bcode_skip();
601:     res = gpio_out(bit, val);
602:     if (res < 0) {
603:         res = -res;
604:         basic_error(res | BASIC_GPIO_ERROR);
605:     }
606:     DMSG(">>> %05d OUT(%d,%d)", Lnum, bit, val);
607: }
608:
609: static void
610: stat_outhz(void)
611: {
612:     word_t bit, val, pull;
613:     int res;
614:
615:     if (bcode_next() != BCODE_LPAREN)
616:         basic_error(BASIC_SYNTAXERROR);
617:     bcode_skip();
618:     bit = eval();
619:     if (bcode_next() != BCODE_COMMA)
620:         basic_error(BASIC_SYNTAXERROR);
621:     bcode_skip();
622:     val = eval();
623:     if (bcode_next() != BCODE_COMMA)
624:         basic_error(BASIC_SYNTAXERROR);
625:     bcode_skip();
626:     pull = eval();
627:     if (bcode_next() != BCODE_RPAREN)
628:         basic_error(BASIC_SYNTAXERROR);
629:     bcode_skip();
630:     res = gpio_outhz(bit, val, pull);
631:     if (res < 0) {
632:         res = -res;
633:         basic_error(res | BASIC_GPIO_ERROR);
634:     }
635:     DMSG(">>> %05d OUTHZ(%d,%d,%d)", Lnum, bit, val, pull);
636: }
637: #endif // PI
638:
639: //
640: // BASIC commands
641: //
642:
643: #define FIRSTCOLSIZE  14
644: static void
645: display_help(char **msg)
646: {
647:     char *ptr;
648:     int cnt = 0, sp, i;
649:
650:     ptr = *msg++;
651:     while (ptr) {
652:         if (cnt == 0)
653:             printf("%s%s%s", ESCAPE_LRED, ptr, ESCAPE_DEFAULT);
654:         else
```

```
655:             printf("%s%s%s", ESCAPE_LGREEN, ptr, ESCAPE_DEFAULT);
656:         sp = FIRSTCOLSIZE - strlen(ptr);
657:         while (sp--)
658:             printf(" ");
659:         if (cnt == 0)
660:             printf("%s", ESCAPE_LRED);
661:         ptr = *msg++;
662:         while (*ptr) {
663:             if (*ptr == '$') {
664:                 printf("\n");
665:                 for (i = 0; i < FIRSTCOLSIZE; i++)
666:                     printf(" ");
667:             } else
668:                 printf("%c", *ptr);
669:             ptr++;
670:         }
671:         if (cnt == 0)
672:             printf("%s", ESCAPE_DEFAULT);
673:         printf("\n");
674:         ptr = *msg++;
675:         cnt++;
676:     }
677: }
678:
679: static void
680: cmd_help(void)
681: {
682:     bcode_t b;
683:     int mode = 0;
684:     char buf[ 8 ], *ptr = buf;
685:
686:     if (bcode_nextisvar()) {
687:         bcode_read(&b);
688:         switch(b.idx) {
689:             case 'C' - 'A':     // Containers
690:                 mode = 1;
691:                 break;
692:
693:             case 'O' - 'A':     // Operators
694:                 mode = 2;
695:                 break;
696:
697:             case 'S' - 'A':     // Statements
698:                 mode = 3;
699:                 break;
700:
701:             case 'B' - 'A':     // BASIC commands
702:                 mode = 4;
703:                 break;
704:
705:             default:            // All
706:                 mode = 5;
707:                 break;
708:         }
709:     }
710:
711:     if (mode == 0) {
712:         printf("---[ Help ]-----------------------------------------------------------
--------\n");
713:         basic_readline("C)ontainers, O)perators, S)tatements, B)ASIC commands, A)ll
(c/o/s/b/a) ? ", buf, sizeof(buf));
714:         printf("-------------------------------------------------------------------
--------\n");
715:         if (*ptr == 'C' || *ptr == 'c')
716:             mode = 1;
717:         else if (*ptr == 'O' || *ptr == 'o')
718:             mode = 2;
719:         else if (*ptr == 'S' || *ptr == 's')
720:             mode = 3;
721:         else if (*ptr == 'B' || *ptr == 'b')
722:             mode = 4;
723:         else
724:             mode = 5;
725:     }
726:
727:     switch (mode) {
728:         case 1:
729:             display_help(help_container);
730:             break;
731:
732:         case 2:
733:             display_help(help_operator);
734:             break;
735:
736:         case 3:
737:             display_help(help_statement);
738:             break;
739:
740:         case 4:
741:             display_help(help_command);
742:             break;
743:
744:         default:
745:             display_help(help_container);
746:             printf("\n");
747:             display_help(help_operator);
748:             printf("\n");
749:             display_help(help_statement);
750:             printf("\n");
751:             display_help(help_command);
752:             break;
753:     }
754: }
755:
756: static void
757: cmd_free(void)
758: {
759:     printf(ESCAPE_LBLUE);
760:     printf("%d lines, %d lines free\n", Lsize, PROG_MAXLINE - Lsize);
761:     printf("%d bytes used, %d bytes free\n", Psize, PROG_MAXSIZE - Psize);
762:     printf(ESCAPE_DEFAULT);
763: }
764:
765: static void
766: cmd_dump(void)
767: {
768:     bcode_t b;
769:
770:     if (bcode_nextisvar()) {
771:         bcode_read(&b);
772:         switch(b.idx) {
773:             case 'V' - 'A':
774:                 cont_dumpvar();
775:                 return;
776:
777:             case 'A' - 'A':
778:                 cont_dumparr();
779:                 return;
780:
781:             case 'S' - 'A':
782:                 cont_dumpstr();
783:                 return;
```

```
784:
785:             case 'B' - 'A':
786:                 prog_dumpbytes();
787:                 return;
788:
789:             case 'L' - 'A':
790:                 prog_dumplines();
791:                 return;
792:         }
793:     }
794:     cont_dumpvar();
795:     printf("\n");
796:     cont_dumparr();
797:     printf("\n");
798:     cont_dumpstr();
799:     printf("\n");
800:     prog_dumpbytes();
801:     printf("\n");
802:     prog_dumplines();
803: }
804:
805: static void
806: cmd_new(void)
807: {
808:     prog_init();
809:     Lnum = Lidx = Gsize = Fsize = 0;
810: }
811:
812: static void
813: cmd_clear(void)
814: {
815:     cont_init();
816: }
817:
818: #ifdef READLINE
819: // EDIT uses the ordinary readline() call with a startup hook that pre-fills
820: // the edit buffer. ^C is therefore handled exactly like at the "> " prompt:
821: // readline restores the terminal itself and re-raises SIGINT to our handler.
822:
823: static char *line_initial;      // text to be inserted when readline() starts
824:
825: static int
826: line_hook(void)
827: {
828:     rl_startup_hook = NULL;     // one-shot: even if ^C aborts this readline(),
829:     if (line_initial) {         // the hook never fires again
830:         rl_insert_text(line_initial);
831:         line_initial = NULL;
832:     }
833:     return 0;
834: }
835:
836: // Render program line "num" to a text buffer (without the trailing newline)
837:
838: static int
839: line_fetch(int num, char *buf, int size)
840: {
841:     FILE *fp;
842:     char *mem = NULL;
843:     size_t len = 0;
844:     int ret;
845:
846:     fp = open_memstream(&mem, &len);
847:     if (fp == NULL)
848:         return -1;
849:     ret = prog_list(fp, PROG_LISTPLAIN, num, num);
850:     fclose(fp);
851:     if (ret == 0) {
852:         if (len > 0 && mem[ len - 1 ] == '\n')
853:             mem[ len - 1 ] = 0;
854:         snprintf(buf, size, "%s", mem);
855:     }
856:     free(mem);
857:     return ret;
858: }
859: #endif
860:
861: static void
862: cmd_edit(void)
863: {
864: #ifndef READLINE
865:     printf("%sError: EDIT command needs GNU Readline Library.%s\n",
866:         ESCAPE_LRED, ESCAPE_DEFAULT);
867:     return;
868: #else
869:     bcode_t b;
870:     int num, ret, len;
871:     char src[ BASIC_MAXLINECHAR ], buf[ BASIC_MAXLINECHAR ], *ptr;
872:     byte_t cmd[ BCODE_MAXSIZE ];
873:
874:     if (bcode_next() != BCODE_NUMBER10)
875:         basic_error(BASIC_NOLINENUM);
876:     bcode_read(&b);
877:     num = b.num;
878:     ret = prog_search(num);
879:     if (ret < 0)
880:         basic_error(BASIC_NOLINENUM);
881:
882:     ret = line_fetch(num, src, sizeof(src));
883:     if (ret < 0)
884:         basic_error(BASIC_FILEIOERROR);
885:     else if (ret > 0)
886:         basic_error(ret | BASIC_PROG_ERROR);
887:
888:     printf("%sTo abort editing, press ^C.%s\n", ESCAPE_LMAGENTA, ESCAPE_DEFAULT);
889:     line_initial = src;
890:     rl_startup_hook = line_hook;
891:     len = basic_readline("> ", buf, sizeof(buf));
892:     if (len <= 0)
893:         return;
894:     ptr = buf;
895:     while (*ptr == ' ')
896:         ptr++;
897:     if (*ptr == 0)
898:         return;
899:
900:     token_source(ptr);
901:     len = bcode_compile(cmd);
902:     if (len < 0) {
903:         printf("%sError: %s\n", ESCAPE_LRED, bcode_result(-len));
904:     } else {
905:         bcode_start(cmd);
906:         bcode_read(&b);
907:         if (b.type == BCODE_TYPE_NUMBER10) {
908:             num = b.num;
909:             if (bcode_next() == BCODE_EOL) {    // --- delete a line
910:                 ret = prog_delete(num, num);
911:                 if (ret) {
912:                     printf("%sError: %s\n", ESCAPE_LRED, prog_result(ret));
913:                 }
914:             } else {                            // --- insert a line
915:                 bcode_read(&b);
```

```
916:                 ret = prog_insert(num, &cmd[ b.pos ], len - 3);
917:                 if (ret) {
918:                     printf("%sError: %s\n", ESCAPE_LRED, prog_result(ret));
919:                 }
920:             }
921:         } else {
922:             printf("%sError: %s\n", ESCAPE_LRED, "Illegal editing");
923:         }
924:     }
925: #endif
926: }
927:
928: static void
929: cmd_list(void)
930: {
931:     bcode_t b;
932:     int start, end, ret;
933:
934:     start = Lines[ 1 ].num;
935:     end = Lines[ Lsize ].num;
936:     if (bcode_next() == BCODE_NUMBER10) {
937:         bcode_read(&b);
938:         start = b.num;
939:         if (start < 1 || start >= PROG_BLANKLINE)
940:             basic_error(BASIC_ILLRANGE);
941:         end = start;
942:         if (bcode_next() == BCODE_COMMA) {
943:             bcode_skip();
944:             if (bcode_next() == BCODE_NUMBER10) {
945:                 bcode_read(&b);
946:                 end = b.num;
947:                 if (end < 1 || end >= PROG_BLANKLINE || start > end)
948:                     basic_error(BASIC_ILLRANGE);
949:             } else
950:                 basic_error(BASIC_ILLRANGE);
951:         }
952:     } else if (bcode_next() != BCODE_EOL)
953:         basic_error(BASIC_ILLRANGE);
954:     ret = prog_list(stdout, PROG_LISTCOLOR, start, end);
955:     if (ret)
956:         basic_error(ret | BASIC_PROG_ERROR);
957: }
958:
959: static void
960: cmd_renum(void)
961: {
962:     bcode_t b;
963:     int start = 0, step = 0, ret;
964:
965:     if (bcode_next() == BCODE_EOL) {
966:         start = BASIC_LINESTART;
967:         step = BASIC_LINESTEP;
968:     } else if (bcode_nextisnum()) {
969:         bcode_read(&b);
970:         start = b.num;
971:         if (bcode_next() == BCODE_COMMA) {
972:             bcode_skip();
973:             if (! bcode_nextisnum())
974:                 basic_error(BASIC_ILLRANGE);
975:             else {
976:                 bcode_read(&b);
977:                 step = b.num;
978:             }
979:         } else if (bcode_next() == BCODE_EOL)
980:             step = BASIC_LINESTEP;
981:         else
982:             basic_error(BASIC_ILLRANGE);
983:     } else
984:         basic_error(BASIC_ILLRANGE);
985:
986:     if (start < 1 || step < 1 || (start + step * Lsize) >= PROG_BLANKLINE)
987:         basic_error(BASIC_ILLRANGE);
988:
989:     ret = prog_renum(start, step);
990:     if (ret)
991:         basic_error(ret | BASIC_PROG_ERROR);
992: }
993:
994: static void
995: cmd_delete(void)
996: {
997:     bcode_t b;
998:     int start = 0, end = 0, ret;
999:
1000:     if (bcode_next() == BCODE_NUMBER10) {
1001:         bcode_read(&b);
1002:         start = b.num;
1003:         if (start < 1 || start >= PROG_BLANKLINE)
1004:             basic_error(BASIC_ILLRANGE);
1005:         end = start;
1006:         if (bcode_next() == BCODE_COMMA) {
1007:             bcode_skip();
1008:             if (bcode_next() == BCODE_NUMBER10) {
1009:                 bcode_read(&b);
1010:                 end = b.num;
1011:                 if (end < 1 || end >= PROG_BLANKLINE || start > end)
1012:                     basic_error(BASIC_ILLRANGE);
1013:             } else
1014:                 basic_error(BASIC_ILLRANGE);
1015:         }
1016:     } else
1017:         basic_error(BASIC_NOLINENUM);
1018:     ret = prog_delete(start, end);
1019:     if (ret)
1020:         basic_error(ret | BASIC_PROG_ERROR);
1021: }
1022:
1023: static void
1024: cmd_files(void)
1025: {
1026:     bcode_t b;
1027:     struct dirent **list;
1028:     char *dir;
1029:     int i, cnt;
1030:
1031:     if (bcode_nextiseol())
1032:         dir = "./";
1033:     else if (! bcode_nextisstr())
1034:         basic_error(BASIC_NODIRNAME);
1035:     else {
1036:         bcode_read(&b);
1037:         dir = (char *) b.str;
1038:     }
1039:     cnt = scandir(dir, &list, NULL, alphasort);
1040:     if (cnt < 0)                            // scandir() returns -1 on error
1041:         basic_error(BASIC_DIRNOTFOUND);
1042:
1043:     for (i = 0; i < cnt; i++) {
1044:         if (list[ i ]->d_name[0] != '.') {
1045:             if (list[ i ]->d_type == DT_DIR) {
1046:                 printf(ESCAPE_LGREEN);
1047:                 printf("%s/\n", list[ i ]->d_name);
```

```
1048:             } else {
1049:                 printf(ESCAPE_LBLUE);
1050:                 printf("%s\n", list[ i ]->d_name);
1051:             }
1052:         }
1053:         free(list[ i ]);                    // each entry is malloc()ed
1054:     }
1055:     printf(ESCAPE_DEFAULT);
1056:     free(list);
1057: }
1058:
1059: static void
1060: cmd_load(void)
1061: {
1062:     bcode_t b;
1063:     FILE *src;
1064:     char *ptr;
1065:     byte_t code[ BCODE_MAXSIZE ];
1066:     int size, line, ret;
1067:
1068:     if (! bcode_nextisstr())
1069:         basic_error(BASIC_NOFILENAME);
1070:     bcode_read(&b);
1071:     printf("Loading file \"%s\"\n", b.str);
1072:     src = fopen((char *) b.str, "r");
1073:     if (src == NULL)
1074:         basic_error(BASIC_FILEIOERROR);
1075:
1076:     prog_init();
1077:     Mode = BASIC_MODELOAD;
1078:     while (1) {
1079:         ptr = fgets(Text, TOKEN_MAXBUF, src);
1080:         if (ptr == NULL)
1081:             break;
1082:         while (*ptr) {
1083:             if (*ptr == '\n') {
1084:                 *ptr = 0;
1085:                 break;
1086:             }
1087:             ptr++;
1088:         }
1089:         token_source(Text);
1090:         size = bcode_compile(code);
1091:         if (size < 0) {
1092:             fclose(src);
1093:             size = -size;
1094:             basic_error(size | BASIC_BCODE_ERROR);
1095:         }
1096:         bcode_start(code);
1097:         if (bcode_nextiseol())          // skip blank lines
1098:             continue;
1099:         if (! bcode_nextisnum()) {
1100:             fclose(src);
1101:             basic_error(BASIC_NOLINENUM);
1102:         }
1103:         bcode_read(&b);
1104:         line = b.num;
1105:         bcode_read(&b);
1106:         ret = prog_insert(line, &code[ b.pos ], size - 3);
1107:         if (ret) {
1108:             fclose(src);
1109:             basic_error(ret | BASIC_PROG_ERROR);
1110:         }
1111:     }
1112:     fclose(src);
1113:     printf("%d lines loaded\n", Lsize);
1114: }
1115:
1116: static void
1117: cmd_merge(void)
1118: {
1119:     bcode_t b;
1120:     FILE *src;
1121:     char *ptr;
1122:     byte_t code[ BCODE_MAXSIZE ];
1123:     int cnt, size, line, ret;
1124:
1125:     if (! bcode_nextisstr())
1126:         basic_error(BASIC_NOFILENAME);
1127:     bcode_read(&b);
1128:     printf("Merging file \"%s\"\n", b.str);
1129:     src = fopen((char *) b.str, "r");
1130:     if (src == NULL)
1131:         basic_error(BASIC_FILEIOERROR);
1132:
1133:     cnt = 0;
1134:     Mode = BASIC_MODELOAD;
1135:     while (1) {
1136:         ptr = fgets(Text, TOKEN_MAXBUF, src);
1137:         if (ptr == NULL)
1138:             break;
1139:         while (*ptr) {
1140:             if (*ptr == '\n') {
1141:                 *ptr = 0;
1142:                 break;
1143:             }
1144:             ptr++;
1145:         }
1146:         token_source(Text);
1147:         size = bcode_compile(code);
1148:         if (size < 0) {
1149:             fclose(src);
1150:             size = -size;
1151:             basic_error(size | BASIC_BCODE_ERROR);
1152:         }
1153:         bcode_start(code);
1154:         if (bcode_nextiseol())          // skip blank lines
1155:             continue;
1156:         if (! bcode_nextisnum()) {
1157:             fclose(src);
1158:             basic_error(BASIC_NOLINENUM);
1159:         }
1160:         bcode_read(&b);
1161:         line = b.num;
1162:         bcode_read(&b);
1163:         ret = prog_insert(line, &code[ b.pos ], size - 3);
1164:         if (ret) {
1165:             fclose(src);
1166:             basic_error(ret | BASIC_PROG_ERROR);
1167:         }
1168:         cnt++;
1169:     }
1170:     fclose(src);
1171:     printf("%d lines merged\n", cnt);
1172: }
1173:
1174: static void
1175: cmd_save(void)
1176: {
1177:     bcode_t b;
1178:     FILE *fp;
1179:     int ret;
```

```
1180:
1181:     if (! bcode_nextisstr())
1182:         basic_error(BASIC_NOFILENAME);
1183:     bcode_read(&b);
1184:     DMSG("Saving file \"%s\"\n", b.str);
1185:     fp = fopen((char *) b.str, "w");        // "w" truncates an existing file
1186:     if (fp == NULL)
1187:         basic_error(BASIC_FILEIOERROR);
1188:     ret = prog_list(fp, PROG_LISTPLAIN, Lines[ 1 ].num, Lines[ Lsize ].num);
1189:     if (ret) {
1190:         fclose(fp);
1191:         basic_error(ret | BASIC_PROG_ERROR);
1192:     }
1193:     if (fclose(fp) != 0)
1194:         basic_error(BASIC_FILEIOERROR);
1195:     printf("%d lines saved\n", Lsize);
1196: }
1197:
1198: static void
1199: deny_direct(void)
1200: {
1201:     if (Mode == BASIC_MODEDIRECT)
1202:         basic_error(BASIC_DIRECTDENY);
1203: }
1204:
1205: int
1206: basic_load(char *fname)
1207: {
1208:     bcode_t b;
1209:     FILE *src;
1210:     char *ptr;
1211:     byte_t code[ BCODE_MAXSIZE ];
1212:     int size, line, ret;
1213:
1214:     src = fopen((char *) fname, "r");
1215:     if (src == NULL)
1216:         return 1;           // 1: File open failure
1217:
1218:     prog_init();
1219:     Mode = BASIC_MODELOAD;
1220:     while (1) {
1221:         ptr = fgets(Text, TOKEN_MAXBUF, src);
1222:         if (ptr == NULL)
1223:             break;
1224:         while (*ptr) {
1225:             if (*ptr == '\n') {
1226:                 *ptr = 0;
1227:                 break;
1228:             }
1229:             ptr++;
1230:         }
1231:         token_source(Text);
1232:         size = bcode_compile(code);
1233:         if (size < 0) {
1234:             fclose(src);
1235:             size = -size;
1236:             return 2;       // 2: Bytecode compile error
1237:         }
1238:         bcode_start(code);
1239:         if (bcode_nextiseol())          // skip blank lines
1240:             continue;
1241:         if (! bcode_nextisnum()) {
1242:             fclose(src);
1243:             return 3;       // 3: Line number undefined
1244:         }
1245:         bcode_read(&b);
1246:         line = b.num;
1247:         bcode_read(&b);
1248:         ret = prog_insert(line, &code[ b.pos ], size - 3);
1249:         if (ret) {
1250:             fclose(src);
1251:             return 4;       // 4: Program error
1252:         }
1253:     }
1254:     fclose(src);
1255:     return 0;
1256: }
1257:
1258: int
1259: basic_exec(void)
1260: {
1261:     bcode_t b;
1262:     int line = 0, jump = 0, back = 0, skip = 0, newpc = 0, var;
1263:
1264:     if (Mode == BASIC_MODEDIRECT)
1265:         line = 0;
1266:     else if (Mode == BASIC_MODERUN) {
1267:         if (Lsize == 0)
1268:             return 0;
1269:         cont_init();
1270:         line = 1;
1271:     } else if (Mode == BASIC_MODESHELL) {
1272:         if (Lsize == 0)
1273:             return 0;
1274:         line = 1;
1275:     }
1276:
1277:     Gsize = Fsize = 0;
1278:     bcode_start(Program);
1279:     while (Lines[ line ].len) {
1280:         Lidx = line;
1281:         Lnum = Lines[ line ].num;
1282: #ifdef DEBUG
1283:         if (Lnum)
1284:             DMSG("Executing line %d", Lnum)
1285:         else
1286:             DMSG("Executing COMMAND line")
1287: #endif // DEBUG
1288:         if (back)
1289:             bcode_setpc(newpc);
1290:         else
1291:             bcode_setpc(Lines[ line ].add);
1292:         jump = back = skip = 0;
1293:         while (1) {
1294:             bcode_read(&b);
1295:             if (b.type == BCODE_TYPE_EOL)
1296:                 break;
1297:             else if (b.inst == BCODE_COLON)
1298:                 continue;
1299:             else if (b.type == BCODE_TYPE_KEYWORD) {
1300:                 switch (b.inst) {
1301:                     case BCODE_END:
1302:                         return(stat_end());
1303:
1304:                     case BCODE_CLS:
1305:                         stat_cls();
1306:                         break;
1307:
1308:                     case BCODE_COLOR:
1309:                         stat_color();
1310:                         break;
1311:
```

```
1312:                     case BCODE_DIM:
1313:                         stat_dim();
1314:                         break;
1315:
1316:                     case BCODE_FOR:
1317:                         deny_direct();
1318:                         stat_for();
1319:                         break;
1320:
1321:                     case BCODE_GOSUB:
1322:                         jump = stat_gosub();
1323:                         break;
1324:
1325:                     case BCODE_GOTO:
1326:                         jump = stat_goto();
1327:                         break;
1328:
1329:                     case BCODE_IF:
1330:                         skip = stat_if() ? 0 : 1;
1331:                         break;
1332:
1333:                     case BCODE_INPUT:
1334:                         stat_input();
1335:                         break;
1336:
1337:                     case BCODE_LOCATE:
1338:                         stat_locate();
1339:                         break;
1340:
1341:                     case BCODE_NEXT:
1342:                         deny_direct();
1343:                         stat_next();
1344:                         var = Fstack[ Fsize - 1 ].vidx;
1345:                         if (Var[ var ] < Fstack[ Fsize - 1 ].ecnt) {
1346:                             Var[ var ]++;       // compare BEFORE increment:
1347:                             line = Fstack[ Fsize - 1 ].lidx;    // no wrap-around
1348:                             newpc = Fstack[ Fsize - 1 ].ladd;   // at 32767
1349:                             back = 1;
1350:                         } else {
1351:                             Var[ var ]++;       // leave end+1 as classic BASIC does
1352:                             Fsize--;
1353:                             back = 0;
1354:                         }
1355:                         break;
1356: #ifdef PI
1357:                     case BCODE_OUT:
1358:                         stat_out();
1359:                         break;
1360:
1361:                     case BCODE_OUTHZ:
1362:                         stat_outhz();
1363:                         break;
1364: #endif // PI
1365:                     case BCODE_PRINT:
1366:                         stat_print();
1367:                         break;
1368:
1369:                     case BCODE_REM:
1370:                         deny_direct();
1371:                         stat_rem();
1372:                         break;
1373:
1374:                     case BCODE_RETURN:
1375:                         deny_direct();
1376:                         stat_return();
1377:                         line = Gstack[ Gsize ].lidx;
1378:                         newpc = Gstack[ Gsize ].radd;
1379:                         back = 1;
1380:                         break;
1381:
1382:                     case BCODE_MSLEEP:
1383:                         stat_msleep();
1384:                         break;
1385:
1386:                     case BCODE_USLEEP:
1387:                         stat_usleep();
1388:                         break;
1389:
1390:                     default:
1391:                         basic_error(BASIC_SYNTAXERROR);
1392:                 } // switch
1393:             } else if (b.type == BCODE_TYPE_VARIABLE)
1394:                 eval_assign(b.idx + 1);
1395:             else if (b.inst == BCODE_AT)
1396:                 eval_assign(0);
1397:             else
1398:                 basic_error(BASIC_SYNTAXERROR);
1399:             if (jump || back || skip)
1400:                 break;
1401:         } // inner while
1402:         if (jump) {
1403:             line = prog_search(jump);
1404:             if (line < 0)
1405:                 basic_error(PROG_LINENOTFOUND | BASIC_PROG_ERROR);
1406:         } else if (back)
1407:             continue;
1408:         else {
1409:             if (line == 0)  // Terminate DIRECT mode
1410:                 return 0;
1411:             line++;
1412:         }
1413:     } // outer while
1414:     return 0;
1415: }
1416:
1417: int
1418: basic_command(int inst)
1419: {
1420:     switch (inst) {
1421:         case BCODE_CLEAR:
1422:             cmd_clear();
1423:             break;
1424:
1425:         case BCODE_CLS:
1426:             stat_cls();
1427:             break;
1428:
1429:         case BCODE_DELETE:
1430:             cmd_delete();
1431:             break;
1432:
1433:         case BCODE_EDIT:
1434:             cmd_edit();
1435:             break;
1436:
1437:         case BCODE_DUMP:
1438:             cmd_dump();
1439:             break;
1440:
1441:         case BCODE_FILES:
1442:             cmd_files();
1443:             break;
```

```
1444:
1445:         case BCODE_FREE:
1446:             cmd_free();
1447:             break;
1448:
1449:         case BCODE_HELP:
1450:             cmd_help();
1451:             break;
1452:
1453:         case BCODE_LIST:
1454:             cmd_list();
1455:             break;
1456:
1457:         case BCODE_LOAD:
1458:             cmd_load();
1459:             break;
1460:
1461:         case BCODE_MERGE:
1462:             cmd_merge();
1463:             break;
1464:
1465:         case BCODE_NEW:
1466:             cmd_new();
1467:             break;
1468:
1469:         case BCODE_RENUM:
1470:             cmd_renum();
1471:             break;
1472:
1473:         case BCODE_RUN:
1474:             Mode = BASIC_MODERUN;
1475:             basic_exec();
1476:             break;
1477:
1478:         case BCODE_SAVE:
1479:             cmd_save();
1480:             break;
1481:
1482:         default:
1483:             return 1;
1484:     }
1485:     return 0;
1486: }
1487:
1488: char *
1489: basic_result(int code)
1490: {
1491:     return Results[ code ];
1492: }
```

```
//
//  p i _ g p i o . h
//
//  BAREMETALHACK.COM --> public domain
//

#ifndef _PI_GPIO_H_
#define _PI_GPIO_H_

// Raspberry Pi SOC type extracted from revision code
// NOTE: pull-up/dowun register handling is different between BCM283x and BCM2711

#define GPIO_REVISIONPATH   "/proc/device-tree/system/linux,revision"
#define GPIO_BCM283x        0       // BCM2835: Pi original, Pi Zero
                                    // BCM2836: Pi 2, Pi 3
#define GPIO_BCM2711        1       // Pi 4, Pi 400
#define GPIO_UPDN_WAIT      1       // Microseconds to wait while pull-up/down
                                    // control in BCM283x
                                    // "BCM2835 ARM Peripheral" says "150cycles"
                                    //    GPIO clock = 250MHz (GPU)
                                    //    150/250M = 0.6 microseconds

// Linux ROOTLESS access to GPIOMEM

#define GPIO_PAGEPATH       "/dev/gpiomem"
#define GPIO_PAGESIZE       4096

// Re-arranged BMH (Bare Metal Hacking) bits: BMH1-BMH14
//      GPIO2, GPIO3, GPIO4, GPIO17, GPIO27, GPIO22, GPIO10, GPIO9, GPIO11,
//      GPIO5, GPIO6, GPIO13, GPIO19, GPIO26

#define GPIO_BITS           14      // BMH1 to BMH14

// GPIO control register index (32bit WORD offset address)

#define FSEL0               0       // Function select 0 (GPIO0 to GPIO9)
#define FSEL1               1       // Function select 1 (GPIO10 to GPIO19)
#define FSEL2               2       // Function select 2 (GPIO20 to GPIO29)
#define SET0                7       // Pin output set 0 (GPIO0 to GPIO31)
#define CLR0                10      // Pin output clear 0 (GPIO0 to GPIO31)
#define LEV0                13      // Pin level 0 (GPIO0 to GPIO31)

// GPIO pull-up/down control registers in BCM2711

#define UPDN0               57      // Pull-up pull-down 0 (GPIO0 to GPIO15)
#define UPDN1               58      // Pull-up pull-down 0 (GPIO16 to GPIO31)

// GPIO pull-up/down control registers in BCM283x

#define GPPUD               37      // Gpio Pull-Up/Down enable register
#define GPPUDCLK0           38      // Gpio Pull-Up/Down enable CLocK 0 register

// Bit values

#define GPIO_HIGH           1
#define GPIO_LOW            0

// Bit definition

#define GPIO_INPUT          0
#define GPIO_OUTPUT         1
#define GPIO_OUTPUTHZ       2
#define GPIO_UNSPECIFIED    (-1)

// Register mode

#define GPIO_REGNONE        0       // NO register is connected
#define GPIO_REGUP          1       // Pull-UP register is connected
#define GPIO_REGDOWN        2       // Pull-DOWN register is connected

// Result codes

#define GPIO_SUCCESS        0
#define GPIO_SOCUNKNOWN     (-1)
#define GPIO_OPENFAIL       (-2)
#define GPIO_ILLBIT         (-3)
#define GPIO_ILLDIR         (-4)
#define GPIO_ILLREG         (-5)

//
// Environment variable which holds calibrated loop count for MICROSECOND wait
//

#define GPIO_MICROENV       "ANT_MICROWAIT"

//
// Function prototypes
//

    int gpio_init(void);
   void gpio_term(void);
    int gpio_bitmode(int, int, int);
    int gpio_bitread(int);
    int gpio_bitwrite(int, int);
    int gpio_in(int);
    int gpio_out(int, int);
    int gpio_outhz(int, int, int);
 char *gpio_result(int);
   void gpio_usleep(int);

#endif // _PI_GPIO_H
```

```
//
//  p i _ g p i o . c
//
//  NOTICE: Supported SOCs are BCM2835, BCM2836, BCM2837, BCM2711
//
//  BAREMETALHACK.COM --> public domain
//

#include <fcntl.h>          // open()
#include <unistd.h>         // close()
#include <sys/mman.h>       // mmap(), munmap()
#include <stdint.h>         // uint32_t
#include <stdlib.h>         // atoi()
#include <unistd.h>         // usleep();
#include "pi_gpio.h"

//
// File scope variables
//

static int Soc;
static volatile uint32_t *Gpio_base;
static uint32_t Save_gpio_fsel0, Save_gpio_fsel1, Save_gpio_fsel2;
static uint32_t Save_gpio_set0, Save_gpio_clr0;
static uint32_t Save_gpio_updn0, Save_gpio_updn1;
static int States[ GPIO_BITS + 1 ];     // indexed by BMH number 1..GPIO_BITS
static int Microsec;

// Bitmapping array: converts from re-arranged BMH (Bare Metal Hacking) number to
// original BCM (Broadcom) number

static int Bitmap[] = {
    0,
    2,      // BMH1  --> GPIO2
    3,      // BMH2  --> GPIO3
    4,      // BMH3  --> GPIO4
    17,     // BMH4  --> GPIO17
    27,     // BMH5  --> GPIO27
    22,     // BMH6  --> GPIO22
    10,     // BMH7  --> GPIO10
    9,      // BMH8  --> GPIO9
    11,     // BMH9  --> GPIO11
    5,      // BMH10 --> GPIO5
    6,      // BMH11 --> GPIO6
    13,     // BMH12 --> GPIO13
    19,     // BMH13 --> GPIO19
    26      // BMH14 --> GPIO26
};

static char *Results[] = {
    "GPIO ok",                      // GPIO_SUCCESS
    "GPIO unknown SOC",             // GPIO_SOCUNKNOWN
    "GPIO open failure",            // GPIO_OPENFAIL
    "GPIO illegal bit number",      // GPIO_ILLBIT
    "GPIO illegal direction",       // GPIO_ILLDIR
    "GPIO illegal register mode",   // GPIO_ILLREG
};

//
// GPIO control functions
//

int
gpio_init(void)
{
    int fd, ret, i;
    uint8_t rev[ 4 ];
    char *ptr;

    fd = open(GPIO_REVISIONPATH, O_RDONLY);
    if (fd < 0)
        return GPIO_SOCUNKNOWN;
    ret = read(fd, rev, sizeof(rev));
    close(fd);
    if (ret != sizeof(rev))
        return GPIO_SOCUNKNOWN;
    switch ((rev[ 2 ] & 0xF0) >> 4) {
        case 0:
        case 1:
        case 2:
            Soc = GPIO_BCM283x;
            break;

        case 3:
            Soc = GPIO_BCM2711;
            break;

        default:
            return GPIO_SOCUNKNOWN;
    };

    fd = open(GPIO_PAGEPATH, O_RDWR | O_SYNC | O_CLOEXEC);  // flags are OR'ed, not
separate args
    if (fd < 0)
        return GPIO_OPENFAIL;

    Gpio_base = (uint32_t *) mmap(NULL, GPIO_PAGESIZE, PROT_READ | PROT_WRITE, \
            MAP_SHARED, fd, 0);
    close(fd);
    if (Gpio_base == MAP_FAILED)
        return GPIO_OPENFAIL;

    for (i = 0; i <= GPIO_BITS; i++)
        States[ i ] = GPIO_UNSPECIFIED;

    Save_gpio_fsel0 = Gpio_base[ FSEL0 ];
    Save_gpio_fsel1 = Gpio_base[ FSEL1 ];
    Save_gpio_fsel2 = Gpio_base[ FSEL2 ];
    Save_gpio_set0  = Gpio_base[ SET0 ];
    Save_gpio_clr0  = Gpio_base[ CLR0 ];
    Save_gpio_updn0 = Gpio_base[ UPDN0 ];
    Save_gpio_updn1 = Gpio_base[ UPDN1 ];

    ptr = getenv(GPIO_MICROENV);
    if (ptr)
        Microsec = atoi(ptr);
    else
        Microsec = 0;

    return GPIO_SUCCESS;
}

void
gpio_term(void)
{
    Gpio_base[ FSEL0 ] = Save_gpio_fsel0;
    Gpio_base[ FSEL1 ] = Save_gpio_fsel1;
    Gpio_base[ FSEL2 ] = Save_gpio_fsel2;
    Gpio_base[ SET0 ]  = Save_gpio_set0;
    Gpio_base[ CLR0 ]  = Save_gpio_clr0;
    Gpio_base[ UPDN0 ] = Save_gpio_updn0;
    Gpio_base[ UPDN1 ] = Save_gpio_updn1;
```

```
132:
133:     munmap((void *) Gpio_base, GPIO_PAGESIZE);
134: }
135:
136: int
137: gpio_bitmode(int bmh, int dir, int reg)
138: {
139:     int bcm, idx;
140:     uint32_t val, mask;
141:
142:     if (bmh < 1 || bmh > GPIO_BITS)
143:         return GPIO_ILLBIT;
144:     if (dir != GPIO_INPUT && dir != GPIO_OUTPUT)
145:         return GPIO_ILLDIR;
146:     if (reg != GPIO_REGNONE && reg != GPIO_REGUP && reg != GPIO_REGDOWN)
147:         return GPIO_ILLREG;
148:
149:     bcm = Bitmap[ bmh ];
150:     idx = FSEL0 + (bcm / 10);
151:     val = Gpio_base[ idx ];
152:     mask = ~(0b111 << (3 * (bcm % 10)));
153:     val &= mask;
154:     if (dir == GPIO_OUTPUT)
155:         val |= (0b001 << (3 * (bcm % 10)));
156:     Gpio_base[ idx ] = val;
157:
158:     if (Soc == GPIO_BCM2711) {
159:         idx = UPDN0 + (bcm / 16);
160:         val = Gpio_base[ idx ];
161:         mask = ~(0b11 << (2 * (bcm % 16)));
162:         val &= mask;
163:         if (reg == GPIO_REGUP)
164:             val |= (0b01 << (2 * (bcm % 16)));
165:         else if (reg == GPIO_REGDOWN)
166:             val |= (0b10 << (2 * (bcm % 16)));
167:         Gpio_base[ idx ] = val;
168:     } else {    // BMC283x
169:         switch (reg) {
170:             case GPIO_REGNONE:
171:                 val = 0b00;
172:                 break;
173:
174:             case GPIO_REGUP:
175:                 val = 0b10;
176:                 break;
177:
178:             case GPIO_REGDOWN:
179:                 val = 0x01;
180:                 break;
181:         };
182:         // The following protocol is adopted from "BCM2835 ARM Peripherals"
183:
184:         Gpio_base[ GPPUD ] = val;               // Update GPPUD
185:         gpio_usleep(GPIO_UPDN_WAIT);            // Wait more than 150 cycles
186:         Gpio_base[ GPPUDCLK0 ] = 1 << bcm;      // Assert clock
187:         gpio_usleep(GPIO_UPDN_WAIT);            // Wait more than 150 cycles
188:         Gpio_base[ GPPUD ] = 0;                 // Remove GPPUD control signal
189:         Gpio_base[ GPPUDCLK0 ] = 0;             // Remove GPPUDCLK0 clock signal
190:     }
191:
192:     return GPIO_SUCCESS;
193: }
194:
195: int
196: gpio_bitread(int bmh)
197: {
198:     int bcm;
199:
200:     if (bmh < 1 || bmh > GPIO_BITS)
201:         return GPIO_ILLBIT;
202:
203:     bcm = Bitmap[ bmh ];
204:     if (Gpio_base[ LEV0 ] & (1 << bcm))
205:         return GPIO_HIGH;
206:     else
207:         return GPIO_LOW;
208: }
209:
210: int
211: gpio_bitwrite(int bmh, int value)
212: {
213:     int bcm, idx;
214:
215:     if (bmh < 1 || bmh > GPIO_BITS)
216:         return GPIO_ILLBIT;
217:
218:     bcm = Bitmap[ bmh ];
219:     if (value)
220:         idx = SET0;
221:     else
222:         idx = CLR0;
223:     Gpio_base[ idx ] = (1 << bcm);
224:
225:     return GPIO_SUCCESS;
226: }
227:
228: int
229: gpio_in(int bmh)
230: {
231:     int ret;
232:
233:     if (bmh < 1 || bmh > GPIO_BITS)     // validate BEFORE indexing States[]
234:         return GPIO_ILLBIT;
235:     if (States[ bmh ] == GPIO_INPUT)
236:         return gpio_bitread(bmh);
237:     else {
238:         ret = gpio_bitmode(bmh, GPIO_INPUT, GPIO_REGUP);
239:         if (ret < 0)
240:             return ret;
241:         else {
242:             States[ bmh ] = GPIO_INPUT;
243:             return gpio_bitread(bmh);
244:         }
245:     }
246: }
247:
248: int
249: gpio_out(int bmh, int val)
250: {
251:     int ret;
252:
253:     if (bmh < 1 || bmh > GPIO_BITS)     // validate BEFORE indexing States[]
254:         return GPIO_ILLBIT;
255:     if (States[ bmh ] == GPIO_OUTPUT)
256:         return gpio_bitwrite(bmh, val);
257:     else {
258:         ret = gpio_bitmode(bmh, GPIO_OUTPUT, GPIO_REGNONE);
259:         if (ret < 0)
260:             return ret;
261:         else {
262:             States[ bmh ] = GPIO_OUTPUT;
263:             return gpio_bitwrite(bmh, val);
```

```
264:         }
265:     }
266: }
267:
268: int
269: gpio_outhz(int bmh, int val, int pull)
270: {
271:     int ret, p;
272:
273:     if (bmh < 1 || bmh > GPIO_BITS)     // validate BEFORE indexing States[]
274:         return GPIO_ILLBIT;
275:     if (val) {
276:         p = pull ? GPIO_REGUP : GPIO_REGNONE;
277:         ret = gpio_bitmode(bmh, GPIO_INPUT, p);
278:         if (ret)
279:             return ret;
280:     } else {
281:         ret = gpio_bitmode(bmh, GPIO_OUTPUT, GPIO_REGNONE);
282:         if (ret)
283:             return ret;
284:         ret = gpio_bitwrite(bmh, 0);
285:         if (ret)
286:             return ret;
287:     }
288:     States[ bmh ] = GPIO_OUTPUTHZ;
289:     return GPIO_SUCCESS;
290: }
291:
292: char *
293: gpio_result(int code)
294: {
295:     return Results[ code ];
296: }
297:
298: void
299: gpio_usleep(int utime)
300: {
301:     int loops;
302:     volatile int i;     // We need "volatile" for prevention of code optimization
303:
304:     if (Microsec == 0) {
305:         usleep(utime);
306:         return;
307:     } else {
308:         loops = utime * Microsec;
309:         for (i = 0; i < loops; i++)
310:             ;
311:     }
312: }
```

```
//
// m a i n . c
//
// BAREMETALHACK.COM --> public domain
//

#include <stdio.h>      // FILE{}, printf()
#include <setjmp.h>     // jmp_buf{}, setjmp()
#include <signal.h>     // sigemptyset(), sigaction(), SIG*, SA_*
#include <stdlib.h>     // atoi()
#include <string.h>     // strncpy()

#include "token.h"
#include "bcode.h"
#include "eval.h"
#include "program.h"
#include "basic.h"
#include "escape.h"
#include "debug.h"

#ifdef PI
#include "pi_gpio.h"
#endif //PI

//
// Global variables
//

FILE *Debug;
sigjmp_buf IntRecovery;
jmp_buf EvalRecovery, BasicRecovery;
int Verbose = 1;

//
// Helper functions
//

static void
welcome(void)
{
    if (Verbose) {
        printf("\n=== Doctor BMH's %s%s%s v%d.%d.%d ===\n", \
                ESCAPE_LGREEN, BASIC_NAME, ESCAPE_DEFAULT, \
                BASIC_MAJOR, BASIC_MINOR, BASIC_PATCH);
        printf("  (enter %sHELP%s for summary, %sEND%s for exit)\n", \
                ESCAPE_LBLUE, ESCAPE_DEFAULT, ESCAPE_LBLUE, ESCAPE_DEFAULT);
        printf("\n");
    }
}

static void
sigint_handler(int sig)
{
    (void) sig;
    siglongjmp(IntRecovery, 1);     // siglongjmp restores the signal mask
}

void
sigint_register(void)
{
    struct sigaction act;

    act.sa_handler = sigint_handler;
    sigemptyset(&act.sa_mask);
    act.sa_flags = 0;               // no SA_RESETHAND: handler stays installed,
                                    // so a 2nd ^C right after the 1st cannot kill u
s
    sigaction(SIGINT, &act, NULL);
}

static void
location(void)
{
    printf("%s", ESCAPE_LGREEN);
    if (Mode == BASIC_MODELOAD)
        printf(" in \"%s\"", Text);
    else if (Mode == BASIC_MODEDIRECT) {
        if (Pnum)
            printf(" in line %d", Pnum);
    } else if (Lnum)
        printf(" in line %d", Lnum);
    printf("%s\n", ESCAPE_DEFAULT);
}

static int
is_command(int inst)
{
    switch (inst) {
        case BCODE_CLEAR:
        case BCODE_CLS:
        case BCODE_DUMP:
        case BCODE_DELETE:
        case BCODE_EDIT:
        case BCODE_FILES:
        case BCODE_FREE:
        case BCODE_HELP:
        case BCODE_LIST:
        case BCODE_LOAD:
        case BCODE_MERGE:
        case BCODE_NEW:
        case BCODE_RENUM:
        case BCODE_RUN:
        case BCODE_SAVE:
            return 1;

        default:
            return 0;
    }
}

//
// Main function
//

int
main(int argc, char *argv[])
{
    char text[ TOKEN_MAXBUF ], *msg;
    byte_t cmd[ BCODE_MAXSIZE ];
    int ret, size, num;
    bcode_t b;
    word_t val;

    // Initializer

    if (argc >= 2)
        Verbose = 0;
    Debug = stderr;
    token_init();
    eval_init(&EvalRecovery);
    cont_init();
    prog_init();
```

```
   132:     basic_init(&BasicRecovery);
   133: #ifdef PI
   134:     ret = gpio_init();
   135:     if (ret < 0) {
   136:         printf("%sCould not identify Raspberry Pi on this system.%s\n", \
   137:             ESCAPE_LRED, ESCAPE_DEFAULT);
   138:         return 255;
   139:     }
   140: #endif // PI
   141:     welcome();
   142:     sigint_register();
   143:
   144:     // === Shell mode ===
   145:
   146:     if (argc >= 2) {                    // Program filename is specified
   147:         if (argc >= 3)                  // --- First number argument is specified
   148:             Var[ 0 ] = atoi(argv[ 2 ]); // --- Store the value in variable 'A'
   149:         if (argc >= 4)                  // --- Second number argument is specified
   150:             Var[ 1 ] = atoi(argv[ 3 ]); // --- Store the value in variable 'B'
   151:         if (argc >= 5) {                // --- String argument is specified
   152:             strncpy((char *) String, (const char *) argv[ 4 ], CONT_MAXSTRING - 1);
   153:             String[ CONT_MAXSTRING - 1 ] = 0;   // strncpy() does not guarantee NUL
   154:         }                               // --- Store the value in string array '@'
   155:         if ((ret = sigsetjmp(IntRecovery, 1)) != 0) {   // --- must precede basic_lo
ad()
   156:             printf("\n%santBASIC: user aborted%s\n", ESCAPE_LRED, ESCAPE_DEFAULT);
   157:             return 2;
   158:         }
   159:         ret = basic_load(argv[ 1 ]);    // --- Load the program
   160:         if (ret) {
   161:             printf("%santBASIC: load error%s\n", ESCAPE_LRED, ESCAPE_DEFAULT);
   162:             return 1;
   163:         }
   164:         if ((ret = setjmp(EvalRecovery)) != 0) {
   165:             printf("%santBASIC: eval() error%s\n", ESCAPE_LRED, ESCAPE_DEFAULT);
   166:             return 3;
   167:         }
   168:         if ((ret = setjmp(BasicRecovery)) != 0) {
   169:             printf("%santBASIC: syntax error%s\n", ESCAPE_LRED, ESCAPE_DEFAULT);
   170:             return 4;
   171:         }
   172:         Mode = BASIC_MODESHELL;         // --- Switch to SHELL mode
   173:         ret = basic_exec();             // --- Execute the program
   174:         return ret;                     // --- return exit status to the shell
   175:     }
   176:
   177:     // Error handler
   178:
   179:     if ((ret = sigsetjmp(IntRecovery, 1)) != 0) {
   180:         if (Lnum) {
   181:             printf("\n%sAborted", ESCAPE_LRED);
   182:             location();
   183:             Lnum = 0;
   184:         } else
   185:             printf("\n");
   186:     }
   187:     if ((ret = setjmp(EvalRecovery)) != 0) {
   188:         printf("%sError: %s", ESCAPE_LRED, eval_result(ret));
   189:         location();
   190:     }
   191:     if ((ret = setjmp(BasicRecovery)) != 0) {
   192:         if (ret & BASIC_BCODE_ERROR) {
   193:             ret -= BASIC_BCODE_ERROR;
   194:             msg = bcode_result(ret);
   195:         } else if (ret & BASIC_PROG_ERROR) {
   196:             ret -= BASIC_PROG_ERROR;
   197:             msg = prog_result(ret);
   198:         }
   199: #ifdef PI
   200:         else if (ret & BASIC_GPIO_ERROR) {
   201:             ret -= BASIC_GPIO_ERROR;
   202:             msg = gpio_result(ret);
   203:         }
   204: #endif
   205:         else
   206:             msg = basic_result(ret);
   207:         printf("%sError: %s", ESCAPE_LRED, msg);
   208:         location();
   209:     }
   210:
   211:     // === Interactive direct mode ===
   212:
   213:     while (1) {
   214:         printf("\e[0;0m");          // Reset screen attributes
   215:         Mode = BASIC_MODEDIRECT;    // Enter direct mode
   216:         Pnum = 0;
   217:         Lnum = 0;                   // Forget the line number of the last run
   218:         ret = basic_readline("> ", text, TOKEN_MAXBUF);
   219:         if (ret < 0)                // EOF (^D or end of redirected input)
   220:             break;
   221:         if (ret == 0)
   222:             continue;
   223:         token_source(text);
   224:         size = bcode_compile(cmd);
   225:         if (size < 0) {
   226:             printf("%sError: %s", ESCAPE_LRED, bcode_result(-size));
   227:             location();
   228:             continue;
   229:         }
   230: #ifdef DEBUG
   231:         bcode_dump(cmd);
   232:         fprintf(Debug, "\n");
   233: #endif // DEBUG
   234:
   235:         bcode_start(cmd);
   236:         bcode_read(&b);
   237:         if (b.type == BCODE_TYPE_NUMBER10) {        // Edit program
   238:             num = b.num;
   239:             if (bcode_next() == BCODE_EOL) {        // --- delete a line
   240:                 ret = prog_delete(num, num);
   241:                 if (ret) {
   242:                     printf("%sError: %s", ESCAPE_LRED, prog_result(ret));
   243:                     location();
   244:                 }
   245:             } else {                                // --- insert a line
   246:                 bcode_read(&b);
   247:                 ret = prog_insert(num, &cmd[ b.pos ], size - 3);
   248:                 if (ret) {
   249:                     printf("%sError: %s", ESCAPE_LRED, prog_result(ret));
   250:                     location();
   251:                 }
   252:             }
   253:             continue;
   254:         } else if (b.inst == BCODE_INQ) {           // Evaluate expression
   255:             if (bcode_next() == BCODE_AT && bcode_twoahead() != BCODE_LBRACK) {
   256:                 bcode_skip();
   257:                 printf("%s%s%s\n", ESCAPE_LBLUE, String, ESCAPE_DEFAULT);
   258:             } else {
   259:                 val = eval();
   260:                 printf("%s%d%s\n", ESCAPE_LBLUE, val, ESCAPE_DEFAULT);
   261:             }
   262:             continue;
```

```
263:         } else if (b.type == BCODE_TYPE_KEYWORD) {
264:             if (b.inst == BCODE_END)                // END
265:                 break;                              // --- exit loop
266:             else if (is_command(b.inst)) {          // BASIC commands
267:                 basic_command(b.inst);              // --- execute it
268:                 continue;
269:             }
270:         }                                           // Direct statement execution
271:         prog_cmdline(cmd, size);                    // --- store one liner program
272:         basic_exec();                               // --- execute it
273:     }
274: #ifdef PI
275:     gpio_term();
276: #endif // PI
277:     return 0;
278: }
```

```c
#include <stdio.h>          // printf()
#include <sys/time.h>       // *_t, timeval{}, gettimeofday()
#include <stdlib.h>         // atoi()

void
microsec_wait(int count)
{
    volatile int i;

    for (i = 0; i < count; i++) {
    };
}

int
main(int argc, char *argv[])
{
    struct timeval start, end;
    time_t sec;
    suseconds_t usec;
    float usecs;
    int count, loops, i;

    if (argc != 3) {
        printf("Usage: antcalib loopcount loops\n");
        return 1;
    }

    count = atoi(argv[ 1 ]);
    loops = atoi(argv[ 2 ]);
    printf("Loopcount = %d\n", count);
    printf("Number of loops = %d\n\n", loops);
    gettimeofday(&start, NULL);
    for (i = 0; i < loops; i++)
        microsec_wait(count);
    gettimeofday(&end, NULL);
    sec = end.tv_sec - start.tv_sec;
    if (end.tv_usec < start.tv_usec) {
        sec--;
        usec = 1000000 + end.tv_usec - start.tv_usec;
    } else
        usec = end.tv_usec - start.tv_usec;
    usecs = sec * 1000000 + usec;
    printf("Elapsed time --> %ld sec %ld usec\n", sec, usec);
    printf("Mean time --> %f usec/loop\n", usecs / loops);
    return 0;
}
```